

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN INFORMATICA

MODERNIZZAZIONE DELLA PIPELINE CI/CD DI UN MONOREPO A MICROSERVIZI: ANALISI D'IMPATTO, BUILD E DEPLOY SELETTIVI PER SERVIZIO

Relatore

Prof. Andrea CALI'

Correlatore

Giovanni MANCINI PhD

Candidato

Giulia CAPUTO

N86003429

Anno Accademico 2025-2026

Abstract

Questa tesi documenta la modernizzazione della pipeline di Continuous Integration e Continuous Delivery di optovera-be, il backend della piattaforma Optovera di SOBEREYE, strutturato come monorepo Maven che raccoglie nove microservizi. Nella configurazione di partenza la pipeline trattava ogni modifica come impatto sull'intero sistema: qualunque cambiamento ricostruiva e ripubblicava le immagini Docker di tutti i servizi, mentre il rilascio in produzione avveniva manualmente, aggiornando a mano i tag delle immagini in un repository Terraform e lanciando un apply. Di conseguenza non era possibile rilasciare in produzione un singolo microservizio in modo indipendente.

Il lavoro introduce un'analisi dell'impatto delle modifiche che, a partire dai file toccati, individua i soli microservizi coinvolti e rende selettivi la build, la containerizzazione e la pubblicazione delle immagini su Amazon ECR, oltre all'analisi statica della qualità con SonarQube. Un refactoring architetturale mirato rimuove l'accoppiamento nel codice condiviso tra microservizi, condizione necessaria affinché la selettività sia reale e concettualmente corretta, non soltanto apparente. L'autenticazione verso AWS è migrata da credenziali statiche a un meccanismo OIDC con credenziali temporanee, e un nuovo job automatizza il deploy in produzione su Amazon ECS, disaccoppiando il ciclo di rilascio applicativo dal provisioning infrastrutturale. Un ulteriore controllo in pipeline presidia la disciplina di un microservizio impattato per singola modifica.

La soluzione è stata integrata e dimostrata con un primo rilascio reale e selettivo nella pipeline di produzione. Sulla pipeline in questione, la selettività riduce significativamente la tempistica della pubblicazione delle immagini di circa del -34% quando un solo microservizio è impattato, e del -92% nello scenario privo di impatto. Vengono infine discussi i limiti residui, tra cui la granularità del rilascio sui push che accorpano più richieste di merge e la persistenza di risorse dati condivise fra i servizi.

Parole chiave: CI/CD, microservizi, monorepo, analisi d'impatto, build selettiva, deploy indipendente, GitLab CI/CD, Docker, Amazon ECS, Terraform, OIDC, DevOps.

Indice

Abstract	i
1 Introduzione	1
1.1 Contesto del progetto e motivazioni	1
1.2 Criticità dell'architettura attuale e del processo di rilascio	2
1.3 Approccio proposto e obiettivi della tesi	3
1.4 Principali risultati conseguiti	4
2 Stato dell'arte e tecnologie	5
2.1 Evoluzione delle architetture software verso i microservizi	5
2.2 DevOps e Continuous Integration / Continuous Delivery	6
2.3 Sicurezza nelle pipeline CI/CD	6
2.4 Tecnologie di containerizzazione e Infrastructure as Code	7
2.5 Pipeline CI/CD nei sistemi distribuiti e strategie di build	8
2.6 Modernizzazione delle pipeline	9
2.7 Tecnologie utilizzate	10
2.7.1 Piattaforma di automazione CI/CD (GitLab CI/CD)	10
2.7.2 Sistema di build e gestione delle dipendenze (Maven)	10
2.7.3 Containerizzazione, registro e orchestrazione dei container (Docker, Amazon ECR, Amazon ECS)	11
2.7.4 Provisioning dell'infrastruttura (Terraform)	11
2.7.5 Analisi statica della qualità del codice (SonarQube)	12
2.8 Metriche di valutazione delle pipeline (DORA Metrics)	12
3 Implementazione della soluzione	13
3.1 Architettura iniziale del sistema	13
3.2 Analisi dei requisiti	14
3.2.1 Pain point e colli di bottiglia rilevati	17
3.3 Progettazione della nuova pipeline	18
3.4 Implementazione della soluzione	20
3.4.1 Analisi delle modifiche (impact detection)	20
3.4.2 Build incrementale	23
3.4.3 Automazione del deploy	24
3.4.4 Containerizzazione dei microservizi	28
3.4.5 Gestione degli artifact	32

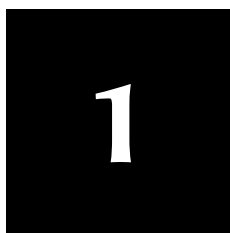
4	Sperimentazione e risultati	33
4.1	Scenario sperimentale	33
4.2	Flusso operativo della pipeline	33
4.2.1	Analisi delle modifiche	33
4.2.2	Build e deploy	35
4.3	Valutazione delle prestazioni	37
4.3.1	Riduzione dei tempi	37
4.3.2	Indipendenza di build e deploy	41
4.4	Sintesi del capitolo	42
5	Considerazioni conclusive	43
5.1	Analisi critica	43
5.1.1	Benefici	43
5.1.2	Limiti	44
5.1.3	Confronto con la soluzione iniziale	46
5.2	Considerazioni finali	48
5.3	Possibili evoluzioni	48
	Glossario	51
	Bibliografia	53
	Testi e articoli scientifici	53
	Documentazione tecnica ufficiale	53
	Fonti interne al progetto (non bibliografiche, per riferimento incrociato)	53

Elenco delle figure

3.1	La pipeline iniziale rappresentata come un "imbuto": qualunque modifica attraversa sempre il percorso completo per tutti i servizi.	15
3.2	Processo di rilascio manuale pre-esistente: aggiornamento a mano del tag nel repository Terraform e terraform apply.	16
3.3	Grafo delle dipendenze fra job (needs:) della pipeline CI/CD.	19
3.4	Pattern a tre rami applicato da ogni job dipendente da IMPACTED_SERVICES: vuoto, ALL o lista di servizi.	20
3.5	Full build (prima) vs. build selettiva (dopo) a fronte della modifica di un solo modulo.	25
3.6	Sequenza di autenticazione OIDC verso AWS: dal JWT firmato da GitLab CI alle credenziali STS temporanee.	26
3.7	Meccanismo di deploy su Amazon ECS tramite revisione immutabile della task definition.	29
3.8	Dispatch dinamico in build_selective: invocazione della funzione di build per ogni servizio impattato.	31
4.1	Traccia end-to-end del rilascio reale in produzione: dal push su main al deploy selettivo su ECS (measurement, web).	37
4.2	Pipeline su main prima dell'introduzione della selettività: 5 job, nessuna selettività, 15 min 50 s complessivi.	38
4.3	Pipeline su main dopo la selettività, nessun microservizio impattato: 8 job, 1 min 49 s complessivi.	38
4.4	Pipeline su main dopo la selettività, un microservizio impattato (measurement): 8 job, 12 min 40 s complessivi.	39
4.5	Pipeline su develop prima dell'introduzione della selettività: 4 job, 14 min 35 s complessivi.	40
4.6	Pipeline su develop dopo la selettività, nessun microservizio impattato: 5 job, 8 min 5 s complessivi.	41

Elenco delle tabelle

4.1	Scenari di verifica dell'analisi d'impatto: modifica introdotta e microservizi rilevati da detect-impact.	34
4.2	Cronologia dei merge confluiti in main nel rilascio reale in produzione. .	35
4.3	Confronto delle durate sulla pipeline di produzione (main).	39
4.4	Confronto delle durate sulla pipeline di staging (develop).	40
5.1	Confronto sintetico fra la pipeline iniziale e la pipeline selettiva.	47



Introduzione

1.1 Contesto del progetto e motivazioni

Optovera è la piattaforma di sicurezza operativa sviluppata da SOBEREYE, azienda di scienza e tecnologia la cui misurazione di riferimento è il riflesso pupillare alla luce (Pupillary Light Reflex, PLR), un segnale fisiologico correlato a fatica, privazione di sonno, stress e assunzione di farmaci. Introdotta nel 2024, la piattaforma viene utilizzata prima dell'inizio del turno di lavoro in attività ad alto rischio (ad esempio nei settori estrattivo e logistico) in oltre dieci paesi, per valutare in modo oggettivo la condizione neurologica del lavoratore. Il backend applicativo che supporta questa piattaforma, e la cui pipeline di rilascio è oggetto di questa tesi, è descritto di seguito.

Il backend di Optovera (nome interno del repository: `optovera-be`, progetto Optovera di SOBEREYE) è strutturato come un **monorepo Maven** basato su Java 21 e Spring Boot, che ospita nove microservizi applicativi distinti — dedicati rispettivamente ad autenticazione, analytics, misurazioni, interfaccia web, gateway, registry, consumer, migrazione e scheduling — oltre a un insieme di moduli condivisi (`opto-core`, `project-model`, `project-service`, tra gli altri) che ne costituiscono le fondamenta comuni.

Il lavoro descritto in questa tesi documenta un percorso che, parte da interventi applicativi puntuali, (correzioni di bug, aggiunta di log per l'osservabilità) per poi convergere su un intervento strutturale effettivo: la riprogettazione della pipeline di integrazione e distribuzione continua (CI/CD) del backend, accompagnata da un refactoring architetturale del codice con un vincolo di processo che impone in modo automatico la disciplina architetturale raggiunta, quella di una modifica per un solo microservizio per merge request. La motivazione di fondo è duplice: da un lato tecnica, un monorepo a microservizi che cresce nel numero di servizi rende insostenibile nel tempo una pipeline pensata per un'applicazione monolitica distribuita; dall'altro organizzativa, rendere il rilascio in produzione un processo affinché sia affidabile, sicuro e ripetibile, riducendo la dipendenza da interventi manuali.

1.2 Criticità dell'architettura attuale e del processo di rilascio

Prima degli interventi descritti in questa tesi, la pipeline CI/CD di optovera-be presentava una struttura **monolitica** distribuita, che comprendeva, indipendentemente dal numero di servizi realmente coinvolti in una modifica:

- **Build sempre completa:** ogni push eseguiva `mvn package` sull'intero reactor Maven, ricompilando tutti i nove microservizi anche quando la modifica riguardava un singolo modulo.
- **Pubblicazione indiscriminata delle immagini Docker:** ad ogni pipeline su main, e quindi in produzione, venivano costruite e pubblicate su Amazon ECR le immagini di *tutti* i servizi, indipendentemente da quali fossero stati effettivamente modificati.
- **Assenza di automazione del deploy in produzione:** l'aggiornamento dei servizi su Amazon ECS non era parte della pipeline, ma richiedeva un intervento manuale o il passaggio attraverso Terraform per ogni singolo rilascio applicativo — un'unione fra il ciclo di vita dell'infrastruttura (per sua natura poco frequente) e quello, molto più frequente, del rilascio di codice applicativo.
- **Autenticazione verso AWS con credenziali statiche:** le pipeline utilizzavano access key e secret key di lunga durata salvate come variabili CI, con la superficie di rischio che questo comporta.
- **Accoppiamento residuo nel codice applicativo:** alcune classi, pur logicamente esclusive di un singolo servizio, risiedevano in moduli Maven condivisi fra più microservizi — una forma di accoppiamento implicito che, a prescindere dalla pipeline, avrebbe reso qualunque strategia di build "selettiva" soltanto apparente.
- **Nessun vincolo di processo sulla dimensione delle modifiche:** nulla, a livello di pipeline, impediva che una singola Merge Request introducesse cambiamenti che attraversavano più microservizi contemporaneamente, vanificando nella pratica il principio di indipendenza fra servizi anche quando l'architettura del codice lo avrebbe permesso.

Da queste caratteristiche derivano criticità concrete e misurabili in termini qualitativi: tempi di pipeline che crescono proporzionalmente al numero di servizi nel monorepo anche per modifiche localizzate; un rischio di regressione incrociata legato alla pubblicazione di immagini non necessarie; un accoppiamento indebito tra il ciclo di rilascio applicativo e uno strumento come Terraform pensato per il provisioning infrastrutturale; un'esposizione di sicurezza legata a credenziali AWS di lunga durata; e, non da ultimo, l'assenza di una garanzia "automatica" che il vincolo di un servizio per modifica venisse effettivamente rispettato nel tempo.

1.3 Approccio proposto e obiettivi della tesi

Il lavoro affronta le criticità individuate attraverso una sequenza di interventi incrementali, ciascuno introdotto, validato empiricamente in pipeline reale e, dove necessario, corretto sulla base di difetti emersi durante la validazione stessa. In ordine cronologico abbiamo:

1. **Analisi d'impatto delle modifiche** (*impact detection*): un job dedicato confronta i file modificati in un push o in una Merge Request con il branch di riferimento e li mappa sui microservizi coinvolti, applicando un criterio conservativo di fallback — qualunque modifica a codice o configurazione condivisa viene trattata come impatto su *tutti* i servizi.
2. **Build Maven selettiva**: il risultato dell'analisi d'impatto viene propagato ai job successivi e utilizzato per limitare la compilazione ai soli moduli effettivamente coinvolti.
3. **Containerizzazione selettiva**: lo stesso principio viene applicato alla costruzione e pubblicazione delle immagini Docker su Amazon ECR.
4. **Refactoring architetturale**: le classi esclusive di un singolo servizio ma collocate in moduli condivisi vengono migrate ai rispettivi moduli applicativi, condizione necessaria affinché la selettività della pipeline sia coerente.
5. **Analisi statica selettiva**: la stessa logica di selettività viene estesa all'analisi di qualità del codice con SonarQube.
6. **Autenticazione AWS via OIDC**: le credenziali statiche vengono sostituite da un meccanismo di autenticazione OIDC, che genera credenziali temporanee per ogni esecuzione di pipeline.
7. **Automazione del deploy in produzione**: la pipeline registra autonomamente nuove revisioni delle task definition ECS e aggiorna i servizi interessati, disaccoppiando il rilascio applicativo dal provisioning infrastrutturale gestito da Terraform.
8. **Introduzione di un vincolo di processo**: un ultimo intervento impone in modo automatico, a livello di pipeline, che una merge request sia responsabile della modifica di un solo microservizio (salvo modifiche deliberate a codice condiviso), impedendo che il disaccoppiamento raggiunto a livello architetturale venga eroso nel tempo da modifiche multi-servizio non controllate.

L'obiettivo della tesi è documentare questo percorso come caso di studio concreto di modernizzazione di una pipeline CI/CD in un contesto monorepo distribuito a microservizi, evidenziando come l'automazione tecnica (punti 1-3, 5-7) da sola non sia sufficiente senza un intervento sul codice applicativo (punto 4) e senza un meccanismo che ne presidi la disciplina nel tempo (punto 8).

È opportuno dichiarare quelle che sono le priorità che guidano l'intero lavoro di tesi, ovvero rendere affidabile, automatico e indipendente il rilascio in **produzione** (punto 7).

Un ambiente di test containerizzato (UAT su Docker) e la raccolta sistematica di metriche di delivery (DORA, Capitolo 2 §2.8) sono traguardi rilevanti e discussi in questa tesi, ma **restano secondari rispetto a tale priorità: la loro assenza, allo stato attuale, non compromette la valutazione complessiva del lavoro svolto.**

1.4 Principali risultati conseguiti

Al termine del percorso descritto, la pipeline di optovera-be presenta le seguenti caratteristiche, riprese e discusse in maggiore dettaglio nei Capitoli 3-5:

- La possibilità, **prima inesistente**, di rilasciare in produzione un singolo microservizio alla volta: conseguenza diretta dell'aver reso selettive analisi d'impatto, build e pubblicazione delle immagini, che ha sciolto il vincolo della build monolitica per cui ogni rilascio muoveva l'intero backend (§3.1).
- Una pipeline **modulare**, in cui build, analisi di qualità, containerizzazione e deploy sono condizionati all'impatto reale di ciascuna modifica, anziché eseguiti indiscriminatamente su tutti i servizi.
- L'eliminazione delle credenziali AWS statiche, sostituite da autenticazione OIDC con credenziali temporanee.
- Un processo di deploy in produzione completamente automatizzato, disaccoppiato dal ciclo di provisioning infrastrutturale.
- Una riduzione dell'accoppiamento a livello di codice fra i microservizi, ottenuta tramite refactoring mirato dei moduli condivisi.
- Un vincolo automatico, applicato in fase di merge request, che impedisce l'introduzione di modifiche che coinvolgono più microservizi contemporaneamente, a presidio nel tempo della disciplina architetturale raggiunta.
- Una riduzione sensibile dei tempi di esecuzione della pipeline nei casi in cui la modifica coinvolge un solo microservizio o nessuno, rispetto all'esecuzione indiscriminata su tutti i nove servizi precedente all'introduzione della selettività (dati reali in Capitolo 4, §4.3.1).

Va precisato che gli ultimi due interventi effettuati, l'automazione del deploy su ECS e l'introduzione del vincolo di processo sulle modifiche multi-servizio, sono stati sviluppati ed effettivamente integrati entrambi nel branch principale (main). Da quel momento la pipeline ha eseguito un primo rilascio reale in produzione in modalità automatica e selettiva, in questo specifico caso, dei servizi measurement e web (Cap. 4, §4.2.2): l'obiettivo primario del lavoro non è quindi solo implementato, ma osservato in esecuzione su Amazon ECS, con i limiti poi discussi in dettaglio nel Capitolo 5, §5.1.2.

2

CAPITOLO 2

Stato dell'arte e tecnologie

2.1 Evoluzione delle architetture software verso i microservizi

L'evoluzione delle architetture software applicative segue, nell'ultimo ventennio, un percorso ben documentato in letteratura: dal **monolite** applicativo, dove l'intera logica di business risiede in un unico deployable, alla **Service-Oriented Architecture (SOA)**, fino all'architettura a **microservizi**, in cui il sistema è scomposto in servizi piccoli, indipendentemente deployabili, organizzati attorno a capacità di business (Newman, *Building Microservices*, 2nd ed.). Un concetto chiave di questa evoluzione è quello di **bounded context**, introdotto da Evans nel contesto del Domain-Driven Design: ogni servizio dovrebbe incapsulare un dominio di responsabilità ben definito, con confini espliciti rispetto agli altri.

Un principio spesso associato ai microservizi, ma non sempre realizzato nella pratica, è quello del **database-per-service**: ogni servizio possiede e gestisce autonomamente il proprio storage, per evitare accoppiamento a livello di schema dati. In *optovera-be* questo principio **trova un'applicazione solo parziale**: dall'ispezione dei profili di produzione dei singoli moduli (`application-prod.yml`) risulta che sei dei nove microservizi (`auth`, `measurement`, `web`, `consumer`, `migration`, `scheduler`) condividono lo stesso database MongoDB, `main_db`, e che `analytics` e `migration` condividono lo stesso database PostgreSQL (`analytics`); anche il broker di messaggi RabbitMQ è condiviso fra più servizi. Lo storage è quindi un punto di accoppiamento tuttora presente fra servizi altrimenti indipendentemente deployabili — un aspetto approfondito nel Capitolo 5, §5.1.2, perché ha una conseguenza diretta sull'analisi d'impatto della pipeline (una modifica ai dati condivisi non viene rilevata come multi-servizio).

Il caso studio di *optovera-be* è rappresentativo di una fase di transizione comune a molti sistemi reali: il backend è organizzato in **nove microservizi**, **ad oggi, indipendentemente deployabili** (autenticazione, analytics, misurazioni, interfaccia web, gateway, registry, consumer, migrazione, scheduling), ma prima degli interventi descritti in questa tesi presentava tipiche caratteristiche di un **"monolita distribuito"**: servizi formalmente separati a runtime, ma accoppiati a livello di build e di codice, attraverso moduli Maven

condivisi (project-service, tra gli altri) che contenevano classi logicamente esclusive di un singolo servizio. Il refactoring descritto nel Capitolo 3 (§3.2, vincolo 4), che migra queste classi verso i rispettivi moduli applicativi, è un intervento concreto di attenuazione di questo pattern anti-architetturale, e costituisce il ponte naturale fra la teoria di questa sezione e l'implementazione discussa nel Capitolo 3.

2.2 DevOps e Continuous Integration / Continuous Delivery

Il movimento DevOps nasce dall'esigenza di superare la separazione tradizionale fra sviluppo (Dev) e operations (Ops), promuovendo pratiche di automazione, collaborazione e miglioramento continuo lungo l'intero ciclo di vita del software (Kim et al., *The DevOps Handbook*). All'interno di questo movimento, **Continuous Integration (CI)** indica la pratica di integrare frequentemente il codice in un ramo condiviso, verificandone automaticamente la qualità (build, test statici e dinamici) ad ogni push; **Continuous Delivery/Deployment (CD)** ne estende il principio fino al rilascio, automatizzando la distribuzione del software in ambienti di staging e, nel caso del *deployment* continuo, direttamente in produzione (Humble & Farley, *Continuous Delivery*).

Il lavoro descritto in questa tesi copre entrambe le dimensioni. Sul fronte CI, la pipeline di *optovera-be* esegue ad ogni push/merge request una sequenza di verifiche automatiche: generazione dell'ambiente Maven (*mvninit*), controllo di formattazione (*spotless-check*), build (*build-maven-project*) e analisi statica (*sonarqube-check*). Sul fronte CD, il deploy verso l'ambiente di staging (*deploy-stage*) e verso la produzione su Amazon ECS (*deploy-prod*, si veda il Capitolo 3 §3.4.3) che automatizzano un processo che in origine richiedeva intervento manuale o il passaggio attraverso strumenti di Infrastructure as Code pensati per un'altra cadenza di rilascio (§2.4). L'automazione del deploy in produzione è stata effettivamente integrata all'interno del branch *main* ed eseguita in un rilascio reale in produzione (Capitolo 4, §4.2.2).

2.3 Sicurezza nelle pipeline CI/CD

Una dimensione della modernizzazione DevOps, meno centrale nel lavoro descritto in questa tesi, riguarda la **sicurezza delle credenziali** utilizzate dalle pipeline per autenticarsi verso servizi esterni. La pratica comunemente diffusa (memorizzare credenziali statiche di lunga durata (access key/secret key) come variabili CI) espone certamente ad un rischio concreto.

L'alternativa moderna, adottata in questo lavoro, è l'autenticazione tramite **OpenID Connect (OIDC) / Workload Identity Federation**: la piattaforma CI (GitLab) emette, per ogni esecuzione di pipeline, un token JWT firmato (*id_tokens.AWS_ID_TOKEN*, *aud: https://sts.amazonaws.com*); il provider cloud (AWS), configurato con un Identity Provider OIDC che si fida dell'issuer GitLab, scambia questo token con credenziali

temporanee tramite il meccanismo `AssumeRoleWithWebIdentity` di AWS STS. Il risultato è l'eliminazione di qualunque segreto di lunga durata dal repository o dalle variabili CI, con credenziali automaticamente scoped alla singola esecuzione di pipeline (`AWS_ROLE_SESSION_NAME` include l'ID pipeline). Questo pattern è oggi documentato come pratica raccomandata sia da GitLab (documentazione ufficiale su "ID tokens") sia da AWS (documentazione su "OIDC identity providers"), ed è coerente con il principio di sicurezza del "least privilege, minimal lifetime".

2.4 Tecnologie di containerizzazione e Infrastructure as Code

Docker fornisce il meccanismo di containerizzazione utilizzato da `optovera-be` per impacchettare ciascun microservizio come immagine autonoma: un'immagine Docker è composta da **layer** immutabili e cacheable, costruiti a partire da un `Dockerfile` tramite istruzioni dichiarative; i **build-arg** permettono di parametrizzare la build dell'immagine senza duplicarne la definizione. Il progetto adotta un **unico Dockerfile radice parametrico**, condiviso da tutti i microservizi: `builder.sh` lo invoca passando di volta in volta `PROJECT_NAME`, `PROJECT_VERSION` e `PROFILE` come build-arg (si veda il Capitolo 3 §3.4.4), così che la stessa definizione produca l'immagine di ciascun servizio. È utile precisare che si tratta di un `Dockerfile single-stage`, non multi-stage: la compilazione Maven avviene a monte, in un job dedicato della pipeline (`build-maven-project`), e il `Dockerfile` impacchetta il JAR in un'immagine runtime leggera basata su Alpine Linux, aggiungendovi l'agent di osservabilità `OpenTelemetry`. Un aspetto sicuramente da migliorare: nella configurazione attuale il JAR viene di fatto ricompilato una seconda volta nel job successivo che costruisce l'immagine, per via delle tempistiche non ancora standardizzate fra i due job: un costo ridondante, oltre che un debito tecnico da risolvere:

```
FROM amazoncorretto:21-alpine
RUN apk add --no-cache --upgrade bash
ARG PROJECT_NAME
ENV PROJECT_NAME=${PROJECT_NAME}
#... altri ARG/ENV: PROJECT_VERSION, PROFILE
COPY ./project-${PROJECT_NAME}/target/project-${PROJECT_NAME}-${PROJECT_VERSION}.jar /opt/project-${PROJECT_NAME}.jar
ARG OTEL_VERSION=2.15.0
ADD https://github.com/open-telemetry/.../v${OTEL_VERSION}/opentelemetry-
    -javaagent.jar otel-agent.jar
ENV JAVA_OPTS="-javaagent:otel-agent.jar..."
ENTRYPOINT [ "/entrypoint.sh" ]
```

La separazione fra la fase di build Maven (in pipeline) e la fase di packaging dell'immagine (nel `Dockerfile`) spiega perché qui non compaia un multi-stage build: il container non ricompila il codice, ne impacchetta solo l'artifact. Va notato (aspetto ripreso nel Capitolo 5, §5.3) che l'agent `OpenTelemetry` viene scaricato da GitHub tramite `ADD` **ad ogni build**

dell'immagine, in assenza di una cache dei layer Docker: un costo fisso ripetuto su ogni rilascio.

Terraform, strumento di Infrastructure as Code (IaC) dichiarativo di HashiCorp, gestisce il *provisioning* dell'infrastruttura cloud (cluster ECS, networking, IAM) attraverso un file di **stato** che rappresenta la configurazione desiderata delle risorse. Il punto rilevante per questa tesi, più che la teoria generale dell'IaC, è l'aspetto effettivamente analizzato nel lavoro: la distinzione fra il ciclo di vita dell'infrastruttura (*provisioning*, a bassa frequenza di modifica) e quello del rilascio applicativo (*deploy* di codice, ad alta frequenza). Utilizzare lo stesso strumento e lo stesso processo per entrambi — come avveniva in *optovera-be* prima dell'introduzione del job *deploy-prod* — introduceva un collo di bottiglia strutturale, poiché il rilascio applicativo ereditava la cadenza e la complessità operativa pensate per un'attività meno frequente: separare i due cicli, restituendo a Terraform il solo ruolo di *provisioning*, è l'obiettivo concreto discusso nel Capitolo 3, §3.4.3.

2.5 Pipeline CI/CD nei sistemi distribuiti e strategie di build

In un sistema distribuito organizzato come monorepo multi-modulo, la strategia con cui la pipeline decide **cosa** costruire ad ogni modifica è una scelta architetturale con impatto diretto su tempi e costi operativi. È utile distinguere quattro approcci, tutti rilevanti per la lettura del lavoro descritto nei capitoli successivi:

- **Full build:** l'intero sistema viene ricompilato/ricostruito ad ogni modifica, indipendentemente da quali file siano cambiati. È la strategia di partenza di *optovera-be* (Capitolo 1, §1.2): semplice da implementare e priva di falsi negativi, ma con un costo che cresce linearmente col numero di moduli/servizi nel monorepo.
- **Incremental build:** si ricostruisce solo ciò che è effettivamente cambiato rispetto all'ultima esecuzione, tipicamente sfruttando cache di compilazione (compilatori incrementali, cache dei task come in Gradle Build Cache). È un concetto affine ma distinto dalla selettività implementata in questo lavoro, che opera a livello di *modulo impattato* più che di singola unità di compilazione.
- **Build selettiva:** la modifica ai file viene analizzata (*diffing*) per determinare quali moduli/servizi sono impattati, e solo questi vengono ricostruiti. È la strategia effettivamente implementata in *optovera-be*: il job *detect-impact* calcola l'insieme dei servizi impattati confrontando i file modificati con un mapping esplicito percorso→servizio, e *build-maven-project* lo traduce in un comando Maven mirato (`mvn package -pl <moduli> -am`).
- **Pipeline modulari:** la pipeline stessa è organizzata in stage e job distinti, con dipendenze esplicite (`needs:`) che permettono esecuzione parallela dove non c'è un vincolo di ordinamento reale, invece di uno script monolitico sequenziale. La pipeline di *optovera-be* adotta questo pattern fin dalla sua struttura a stage (`init`

→ lint → build → sonarqube → test → deploy → images, si veda il Capitolo 3 §3.3).

Un aspetto da tenere presente in vista del Capitolo 5: esistono strumenti dedicati al problema della build selettiva in monorepo come ad esempio **Nx** (con il concetto di *affected projects*), **Bazel** (con `bazel query` e la sua cache remota basata su hash del grafo di dipendenze) e **Turborepo**, che calcolano il grafo delle dipendenze in modo automatico e tipicamente a grana più fine rispetto a un mapping percorso→servizio mantenuto a mano. Maven stesso offre, tramite il **Reactor** multi-modulo e le opzioni `-pl/-am`, un meccanismo nativo di selettività di cui `optovera-be` si avvale direttamente, ma la *decisione* di quali moduli passare a `-pl` resta, in questo progetto, un calcolo esterno e manuale (`detect-impact`), non un'analisi automatica del grafo delle dipendenze — un compromesso pragmatico, adeguato alla scala attuale di nove servizi, e un aspetto su cui riflettere per sviluppi futuri (Capitolo 5, §5.1.2).

2.6 Modernizzazione delle pipeline

Il percorso di modernizzazione di una pipeline CI/CD in un contesto monorepo a micro-servizi può essere descritto come una progressione:

Pipeline monolitiche → **pipeline modulari** → **pipeline "per servizio"**. `optovera-be` si colloca oggi nel secondo stadio di questa progressione: la pipeline è **un'unica definizione YAML condivisa** (`.gitlab-ci.yml`), i cui job sono però condizionati, tramite `IMPACTED_SERVICES`, all'impatto reale della modifica — build, analisi di qualità, containerizzazione e deploy vengono eseguiti solo per i servizi coinvolti. È una distinzione concettuale importante da mantenere esplicita nella tesi: questo **non** equivale a "pipeline indipendenti per servizio" nel senso di un pattern poly-repo, dove ogni microservizio vive nel proprio repository con la propria definizione di pipeline completamente separata — è piuttosto **build/deploy selettivi all'interno di un monorepo**, un pattern architetturealmente diverso con trade-off diversi. La scelta di restare in monorepo, invece di adottare un pattern poly-repo, è stata valutata esplicitamente durante la fase di analisi architetturale che ha preceduto l'implementazione, privilegiando la semplicità operativa alla scala attuale di nove servizi rispetto all'overhead di versionamento cross-repo che un poly-repo introdurrebbe per i moduli condivisi.

Independent deployment: la possibilità di rilasciare un singolo microservizio senza coinvolgere gli altri è resa possibile, nel lavoro descritto, dalla combinazione di tre fattori: (a) build e immagini Docker prodotte per singolo servizio; (b) un job di deploy (`deploy-prod`) che itera sui servizi impattati e aggiorna un ECS service alla volta, indipendentemente dagli altri otto; (c) il refactoring architetturale che riduce le dipendenze condivise, condizione necessaria perché l'indipendenza a livello di deploy non nasconda un accoppiamento ancora presente a livello di codice.

Artifact indipendenti: coerentemente con il principio precedente, ogni servizio produce un proprio artifact Maven (`project-<servizio>-boot/ target/*.jar`) e una propria immagine Docker (`optovera-<servizio>`, taggata con `${CI_COMMIT_REF_SLUG}-${CI_COMMIT_SHORT_SHA}`), pubblicata su un repository ECR dedicato — non un singolo artifact monolitico condiviso da tutti i servizi.

Problemi del monorepo: la scelta di mantenere un monorepo, invece di migrare verso un pattern poly-repo, comporta un compromesso da presentare in modo bilanciato. Fra i vantaggi tipicamente citati in letteratura (Potvin & Levenberg, *Why Google Stores Billions of Lines of Code in a Single Repository*, CACM 2016): visibilità end-to-end sull'intero codice, possibilità di refactoring atomici che attraversano più moduli in un unico commit, un solo storico di versionamento da mantenere coerente. Fra gli svantaggi: senza un meccanismo di selettività, i tempi di build/CI crescono con l'intero repository anche per modifiche piccole; il rischio di accoppiamento implicito fra moduli è strutturalmente più alto che in un poly-repo, dove i confini fisici fra repository impongono una separazione esplicita; l'infrastruttura CI necessaria per mitigare questi svantaggi (il sistema di *impact detection* costruito in questo lavoro) è essa stessa un costo di sviluppo e manutenzione non banale.

2.7 Tecnologie utilizzate

2.7.1 Piattaforma di automazione CI/CD (GitLab CI/CD)

GitLab CI/CD è la piattaforma di automazione utilizzata da `optovera-be`, configurata interamente tramite il file dichiarativo `.gitlab-ci.yml`. I concetti GitLab CI più rilevanti per questo lavoro sono: gli `stages` (fasi sequenziali della pipeline), i `job` (unità di esecuzione all'interno di uno stage), `needs`: che permette di dichiarare dipendenze dirette fra job, abilitando esecuzione parallela indipendentemente dall'ordine degli stage — il meccanismo di **DAG execution** ripreso nel Capitolo 3, §3.3), gli `artifacts` in particolare gli artifact di tipo `reports.dotenv`, usati per propagare `IMPACTED_SERVICES` come variabile d'ambiente fra job, si veda il Capitolo 3 §3.4.5), le regole `rules/only` che condizionano l'esecuzione di un job a un ref specifico, e gli `id_tokens`, il meccanismo che abilita l'autenticazione OIDC discussa in §2.3.

2.7.2 Sistema di build e gestione delle dipendenze (Maven)

Apache Maven gestisce build e dipendenze del backend attraverso un **Reactor multi-modulo**: il `pom.xml` radice dichiara 15 moduli (`<module>`), di cui nove corrispondono a microservizi deployabili e i restanti a librerie/moduli condivisi. Le opzioni `-pl` (*projects list*, limita la build ai moduli indicati) e `-am` (*also make*, include automaticamente le dipendenze upstream nel reactor) sono il meccanismo nativo che rende possibile la build selettiva descritta in §2.5. Maven è utilizzato anche per l'enforcement dello stile del codice

(`mvn spotless:check`, `job spotless-check`) e come punto di integrazione per il plugin SonarQube (§2.7.5).

2.7.3 Containerizzazione, registro e orchestrazione dei container (Docker, Amazon ECR, Amazon ECS)

Le tecnologie descritte in questo paragrafo coprono tre fasi consecutive del ciclo di vita di un'immagine container: la costruzione (Docker), la pubblicazione su registro (Amazon ECR) e l'esecuzione in produzione (Amazon ECS); si veda §2.4 per l'inquadramento concettuale della containerizzazione. Nel progetto la costruzione delle immagini è orchestrata da `builder.sh`, script `bash` che definisce funzioni riutilizzabili (`build_image`, `build_selective`) e `astrae` la costruzione rispetto al `job` GitLab CI che le invoca, rendendole eseguibili anche in locale per debug (Capitolo 3 §3.4.4). Le immagini così prodotte vengono pubblicate su Amazon Elastic Container Registry (ECR), il registro privato che ospita un repository dedicato per ciascun microservizio (`ECR_CONSUMER`, `ECR_GATEWAY`, `ECR_AUTH`, ecc.); l'autenticazione verso ECR avviene tramite `aws ecr get-login-password`, nel contesto delle credenziali temporanee ottenute via OIDC (§2.3).

Amazon Elastic Container Service (ECS) è il servizio utilizzato per orchestrare l'esecuzione dei container in produzione, con `launch type` `EC2`: i container girano su istanze `EC2` gestite direttamente, non su infrastruttura `serverless` `Fargate`. Il pattern di `deploy` implementato (`job deploy-prod`, Capitolo 3 §3.4.3) segue la pratica canonica delle **task definition immutabili**: anziché modificare una configurazione esistente (le `task definition` ECS sono immutabili per design), la pipeline ne registra una **nuova revisione**, clonando quella corrente e sostituendo solo il riferimento all'immagine Docker, per poi aggiornare il service ECS corrispondente (`aws ecs update-service`), che gestisce autonomamente un *rolling deployment* dei nuovi task. Un limite da segnalare: la pipeline non gestisce oggi un percorso esplicito di `rollback` verso la revisione precedente in caso di malfunzionamento della nuova versione: la vecchia `task definition` resta disponibile a livello ECS, ma il suo ripristino richiederebbe un intervento manuale, un limite da tenere presente per gli sviluppi futuri. Ogni microservizio corrisponde a un cluster ECS dedicato (`optovera-<servizio>-prod`), un dettaglio di naming discusso più avanti, nel Capitolo 5, §5.1.2, a proposito del rischio di accoppiamento implicito col repository Terraform.

2.7.4 Provisioning dell'infrastruttura (Terraform)

Si veda §2.4 per la trattazione concettuale. Il repository che contiene le configurazioni Terraform effettive (`terraform-optovera`) è distinto dal backend applicativo e non è stato ispezionato nell'ambito di questa tesi: le affermazioni su Terraform in questo capitolo si limitano a quanto osservabile indirettamente dal lato `optovera-be` (in particolare, dalla documentazione interna relativa al disaccoppiamento del rilascio applicativo discusso nel Capitolo 3).

2.7.5 Analisi statica della qualità del codice (SonarQube)

SonarQube (nella sua declinazione cloud, SonarCloud) esegue analisi statica del codice — code smell, vulnerabilità, duplicazione, copertura dei test — applicando un **Quality Gate**: una soglia configurabile che, in teoria, dovrebbe impedire la promozione di codice che non la supera. Nel progetto, l'analisi è integrata tramite il plugin Maven (sonar-maven-plugin) nel job sonarqube-check, reso selettivo con la stessa logica di impatto descritta in §2.5: la selettività incide qui soprattutto sui tempi di esecuzione del job, riducendoli in modo significativo rispetto all'analisi indiscriminata dell'intero reactor.

2.8 Metriche di valutazione delle pipeline (DORA Metrics)

Le **DORA metrics** (Forsgren, Humble & Kim, *Accelerate: The Science of Lean Software and DevOps*), sviluppate a partire dalla ricerca del team DevOps Research and Assessment di Google, costituiscono il framework più diffuso per misurare le prestazioni di delivery di un'organizzazione software attraverso quattro indicatori:

- **Deployment Frequency**: con quale frequenza un'organizzazione rilascia in produzione.
- **Lead Time for Changes**: il tempo che intercorre fra un commit e la sua esecuzione in produzione.
- **Change Failure Rate**: la percentuale di rilasci che causano un fallimento in produzione (richiedono hotfix, rollback, patch).
- **Time to Restore Service (MTTR)**: il tempo necessario a ripristinare il servizio dopo un incidente causato da un rilascio.

Queste quattro metriche costituiscono il framework di valutazione teorico adottato nel Capitolo 4 per discutere l'impatto, atteso o osservato, del lavoro descritto in questa tesi: build/deploy selettivi e automazione del rilascio agiscono principalmente su Deployment Frequency (attesa in aumento) e Lead Time for Changes (atteso in calo); il vincolo di processo introdotto nell'ultima fase del lavoro (Capitolo 3 §3.4.1-continua) è motivabile in termini di Change Failure Rate, riducendo il "blast radius"(raggio d'azione) di una singola modifica; il deploy automatizzato incide potenzialmente anche sul tempo di ripristino (MTTR), pur non essendo stato progettato specificamente per quello scopo. È importante segnalare fin da questa sezione, per coerenza con quanto già discusso nel Capitolo 4 che, allo stato attuale, **nessuna di queste quattro metriche è raccolta o monitorata in modo strumentato** nella pipeline: la loro trattazione in questa tesi resta un framework di valutazione qualitativo e teorico, finché tale raccolta non verrà poi effettivamente implementata.

3

CAPITOLO 3

Implementazione della soluzione

3.1 Architettura iniziale del sistema

Come introdotto nel Capitolo 1, *optovera-be* è un monorepo Maven organizzato in 15 moduli, di cui nove corrispondono a microservizi indipendentemente deployabili. All'inizio del percorso descritto in questa tesi, tuttavia, questa indipendenza esisteva solo a livello di codice: la pipeline CI/CD trattava ogni modifica come se riguardasse l'intero sistema.

- `build-maven-project` eseguiva incondizionatamente `mvn package` sull'intero reactor Maven, indipendentemente da quale modulo fosse stato effettivamente modificato.
- `build-and-push-images` costruiva e pubblicava su registry le immagini Docker di *tutti* i servizi ad ogni pipeline eseguita su `main`.
- Non esisteva alcun job di deploy automatico verso la produzione: era presente solo `deploy-stage`, un deploy dell'intera applicazione verso un singolo ambiente di staging condiviso, via SSH.
- L'autenticazione verso AWS avveniva tramite credenziali statiche memorizzate come variabili CI.

Il rilascio in produzione avveniva quindi al di fuori della pipeline, come procedura manuale che attraversava il repository dedicato di Terraform. Il flusso ricostruito, era il seguente:

- una pipeline su `main` costruiva e pubblicava le immagini Docker di *tutti* i servizi, contrassegnandole con un tag derivato dallo SHA del commit;
- lo SHA veniva letto manualmente dal log del job di build eseguito su `main`;
- lo si riportava a mano nel file di variabili locali del repository Terraform (`terraform-optovera`), che associa a ciascun servizio il tag dell'immagine da rilasciare;

- si lanciava infine `terraform apply`, che allineava l'infrastruttura ECS alle immagini così indicate. Nella fase di transizione, prima che il flusso fosse interamente automatizzato in pipeline, questo passaggio manuale veniva applicato limitatamente ai soli microservizi effettivamente impattati dalla modifica, di cui si aggiornava manualmente il tag nel file locale, uno alla volta.

È doveroso specificare che il repository Terraform, **non è stato ispezionato** durante il lavoro di tesi. Distinto dal backend applicativo, quello che ci interessa è il *ruolo* nel flusso di rilascio, non la struttura interna. Il punto rilevante, ed è la ragione per cui **prima di questo lavoro non era possibile rilasciare un singolo microservizio**, era che il blocco non risiedeva nel passaggio manuale, ma era a monte all'interno del processo di build. Poiché ogni pipeline su main ricostruiva e ripubblicava le immagini di tutti e nove i microservizi con lo stesso tag, ogni `terraform apply` finiva per allineare l'intera applicazione alla stessa nuova build. Il deploy del singolo servizio non era dunque una funzione mancante del passo di rilascio, ma una possibilità preclusa a monte dalla natura monolitica della stessa: rendere selettive l'analisi d'impatto, la build e la pubblicazione delle immagini (§§3.4.1-3.4.4) è precisamente ciò che, come effetto, l'ha resa possibile. La Figura 3.1 rappresenta visivamente questo comportamento a "imbuto".

È utile rappresentare questa configurazione iniziale come una pipeline "**a imbuto**": qualunque fosse il contributo reale della modifica, anche un singolo file in un singolo microservizio, il flusso di lavoro convergeva sempre sullo stesso percorso completo (build → immagine → deploy) per tutti e nove i microservizi. Il processo manuale di rilascio appena descritto è rappresentato da un diagramma di sequenza dedicato (Figura 3.2), da confrontare poi più avanti con lo schema a grafo della pipeline selettiva finale (§3.3) e con la traccia del rilascio automatico realmente eseguito in produzione (§4.2.2).

3.2 Analisi dei requisiti

Un elemento rilevante, riscontrabile direttamente nel codice, è che il primo intervento della pipeline (il job `detect-impact`) documenta i propri requisiti funzionali sotto forma di criteri di accettazione in stile **Given/When/Then**, direttamente nei commenti dello script:

```
CRITERIA 1: "Given that the pipeline is triggered by a push or merge ←
  request"
CRITERIA 2: "When the analysis stage is reached, Then the modified files←
  are identified"
CRITERIA 3: "...and mapped to their respective microservices"
CRITERIA 4: "...and the list of impacted services is printed to the ←
  console."
```

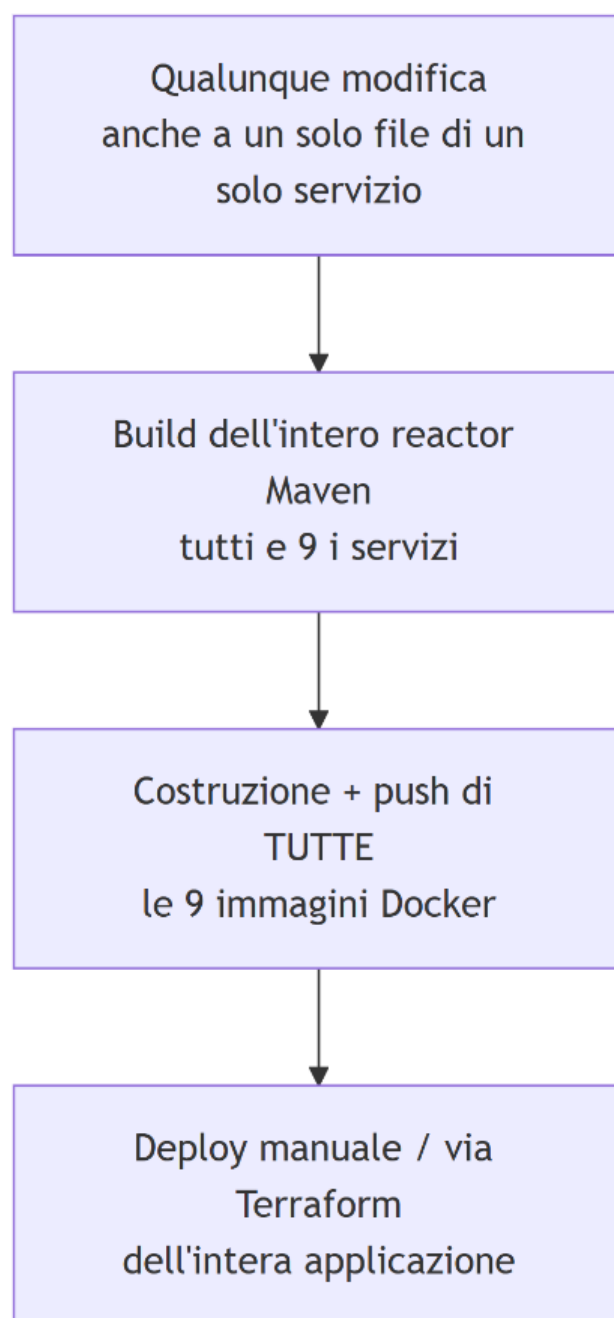


Figura 3.1: La pipeline iniziale rappresentata come un "imbuto": qualunque modifica attraversa sempre il percorso completo per tutti i servizi.

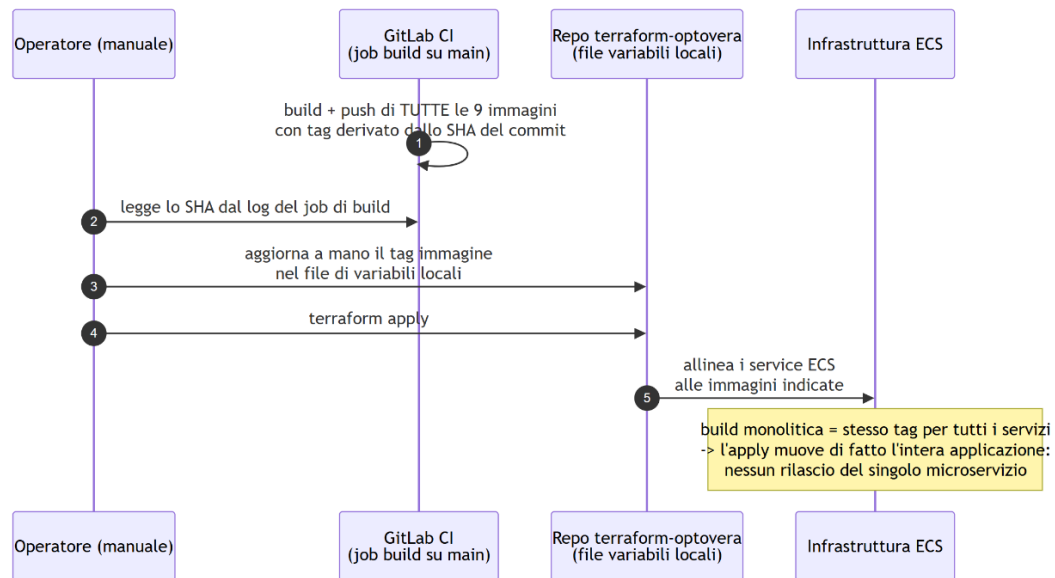


Figura 3.2: Processo di rilascio manuale pre-esistente: aggiornamento a mano del tag nel repository Terraform e terraform apply.

Questo dettaglio, per quanto minimale, testimonia un approccio allo sviluppo guidato da criteri di accettazione espliciti, coerente con la pratica, (Capitolo 4) di validare ogni comportamento atteso con scenari reali in pipeline prima di considerarlo concluso.

Dall'insieme degli interventi affrontati nel percorso (Capitolo 1, §1.3) è possibile ricostruire un insieme coerente di vincoli progettuali che hanno guidato le scelte implementative discusse nel resto del capitolo:

- **Mantenere il funzionamento della pipeline esistente durante lo sviluppo incrementale:** quasi tutti i job mantengono only: refs: [develop, main], con merge_requests aggiunto e rimosso nel corso del lavoro per verificare puntualmente il corretto comportamento della pipeline in fase di sviluppo.
- **Fallback sicuro a "tutto impattato"** in presenza di modifiche a codice condiviso (opto-core/, pom.xml): una scelta conservativa che privilegia l'assenza di falsi negativi (un servizio realmente impattato ma non ricostruito) a costo di una minore selettività nei casi limite, discusso in dettaglio in §3.4.1.
- **Propagazione dello stato fra job di stage diversi**, che in GitLab CI sono per default isolati fra loro: risolto tramite l'uso di un artifact "dotenv" (§3.4.5).
- **Riduzione dell'accoppiamento reale nel codice**, non solo nella pipeline: il refactoring che migra le classi esclusive di un singolo servizio dal modulo condiviso project-service verso i rispettivi moduli -boot è un prerequisito logico, non solo un miglioramento, perché la selettività della build sia *corretta* e non solo apparente:

se due servizi condividono una o più classi collocate in un modulo comune, una build selettiva rischia di non ricompilare codice che, nei fatti, è condiviso fra loro.

- **Eliminazione delle credenziali statiche** (vincolo di sicurezza discusso in §3.4.3).
- **Disaccoppiamento del rilascio applicativo da Terraform** (vincolo architetturale discusso in §3.4.3).

3.2.1 Pain point e colli di bottiglia rilevati

Una valutazione della pipeline attuale, condotta a supporto del lavoro ingegneristico che ha preceduto e accompagnato questo sviluppo, mette in evidenza una serie di colli di bottiglia **tuttora aperti**, distinti dai vincoli progettuali già affrontati. La lettura di questi limiti va fatta alla luce di una priorità dichiarata esplicitamente per questo lavoro: l'obiettivo fondamentale è il contributo per la **produzione** : un deploy automatico, indipendente e affidabile su Amazon ECS, mentre un ambiente UAT containerizzato e la raccolta di metriche DORA, per quanto rilevanti, sono obiettivi **secondari e non bloccanti** per il lavoro di tesi.

Bottleneck sul percorso critico verso la produzione:

- **Nessuna cache delle dipendenze Maven**: nessun job dichiara un blocco cache: per la directory `.m2/repository` — ogni esecuzione di `build-maven-project` e `sonarqube-check` riscarica l'intero albero delle dipendenze da zero, indipendentemente dal fatto che la build sia selettiva o meno, allungando direttamente il tempo fra commit e rilascio in produzione.
- **Nessuna cache dei layer Docker** nella funzione `build_image()` di `builder.sh`, ogni immagine destinata alla produzione parte "a freddo".
- **Test end-to-end non selettivi**: il job `api-test` esegue l'intera collezione Postman a prescindere dal servizio effettivamente impattato, solo su `main`, e con `allow_failure: true`, per cui non blocca un rilascio problematico prima che raggiunga la produzione, un aspetto rilevato ma non approfondito in questo lavoro, che si è concentrato sulla catena di build e deploy piuttosto che sulla suite di test end-to-end.
- **Quality Gate SonarQube non enforced**: anche `sonarqube-check` ha `allow_failure: true` e viene eseguito solo su `main`, dopo il merge, una violazione della qualità del codice, oggi, non blocca un rilascio in produzione.
- **Su main, la selettività ha la granularità del push, non della Merge Request**: gli ultimi due interventi (deploy automatico, §3.4.3, e gate di rifiuto multi-servizio, §3.4.1-continua) sono stati integrati entrambi in `main`. Resta però un collo di bottiglia più sottile, emerso proprio dal primo rilascio reale (Cap. 4, §4.2.2): su `main`, `detect-impact` calcola l'impatto sull'intero arco di un push (`$CI_COMMIT_BEFORE_SHA..$CI_COMMIT_SHA`), per cui più Merge Request

confluite in un unico avanzamento del ramo producono un `IMPACTED_SERVICES` che è la loro unione, e `deploy-prod` rilascia più servizi in un'unica esecuzione. Il gate `reject-multi-service-impact`, girando `only: merge_requests`, non interviene su `main`. È il punto più rilevante di questa lista rispetto all'obiettivo dichiarato ora che l'integrazione è avvenuta, ed è discusso in dettaglio nel Capitolo 5, §5.1.2.

Bottleneck secondari (riconosciuti, non bloccanti rispetto all'obiettivo primario):

- **Nessun ambiente UAT containerizzato:** `deploy-stage` resta un deploy via SSH dell'intera applicazione su un host di staging condiviso, non basato su Docker, non isolato per servizio — un gap riconosciuto, ma non necessario per valutare il successo del lavoro orientato alla produzione.
- **Nessuna raccolta di metriche DORA** strumentata in pipeline: la trattazione teorica del Capitolo 2, §2.8, resta per ora un framework di valutazione qualitativo, non un cruscotto alimentato da dati reali: utile, ma non strettamente necessaria rispetto all'obiettivo primario.

Questi limiti vengono ripresi con spirito costruttivo nel Capitolo 5 e non vanno letti come un giudizio negativo sul lavoro svolto, quanto come una mappa quanto più onesta di ciò che resta da completare rispetto all'obiettivo complessivo dell'iniziativa aziendale in cui questo lavoro si inserisce (epic "Microservices Decoupling & DevOps Pipeline Modernization").

3.3 Progettazione della nuova pipeline

La pipeline, nella sua configurazione più recente osservabile nel repository, è organizzata negli stage seguenti:

```
init → lint → build → sonarqube → test → deploy → images → deploy-prod
```

Lo stage `deploy-prod` è stato introdotto e integrato in `main` (si veda §3.4.3); nella stessa fase è stato introdotto nella pipeline anche il gate `reject-multi-service-impact` (§3.4.1-continua).

Più che la sequenza di stage in sé, l'aspetto architetaturalmente rilevante è il **grafo di dipendenze esplicite fra job** (`needs:`), che permette a GitLab CI di eseguire in parallelo job che non hanno un vincolo di ordinamento reale, invece di rispettare rigidamente l'ordine degli stage:

```
mvninit _____ ^
detect-impact —+—> spotless-check
```

```

+----> build-maven-project ----> build-and-push-images ----> deploy-↔
      prod
└----> sonarqube-check

```

Il job `detect-impact` viene eseguito nello stage `init`, **in parallelo** rispetto a `mvninit`; i job a valle dichiarano esplicitamente `needs: [mvninit, detect-impact]` per poter leggere sia le credenziali Maven (`.m2/settings.xml`) sia il risultato dell'analisi d'impatto (`IMPACTED_SERVICES`). Questo è, concretamente, il meccanismo di **DAG execution** alla base della decisione architetturale di restare in monorepo con analisi d'impatto dinamica invece di adottare un pattern poly-repo (si veda anche il Capitolo 2, §2.6). Il grafo completo delle dipendenze fra job è riportato in Figura 3.3.

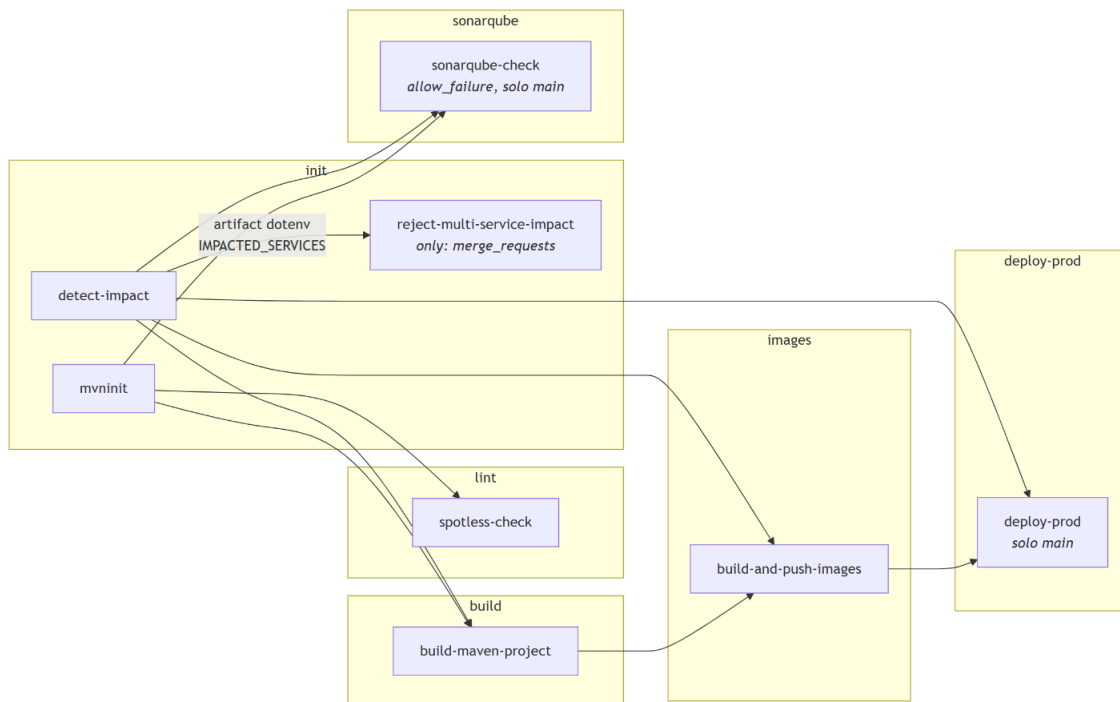


Figura 3.3: Grafo delle dipendenze fra job (needs:) della pipeline CI/CD.

Un principio di design che ricorre identico in più punti della pipeline è quanto si può definire un **pattern a tre rami**, applicato da ogni job che dipende dal risultato dell'analisi d'impatto:

```

if [ -z "$IMPACTED_SERVICES" ]; then
  echo "Nessun servizio impattato. Skip."; exit 0
elif [ "$IMPACTED_SERVICES" = "ALL" ]; then
  # esegue l'operazione su tutto il reactor / tutti i servizi
else
  # esegue l'operazione solo sui servizi in $IMPACTED_SERVICES

```

```
fi
```

Questo pattern compare, con variazioni minime, in `build-maven-project`, `sonarqube-check`, `build-and-push-images` e `deploy-prod`: è una scelta di design coerente, che rende ogni job autonomamente in grado di interpretare correttamente lo stato "nessun impatto / impatto totale / impatto parziale, esaminato in dettaglio nel Capitolo 5, di essere ripetuta letteralmente invece che fattorizzata in un template comune. La Figura 3.4 schematizza il pattern a tre rami applicato da ogni job che dipende da `IMPACTED_SERVICES`.

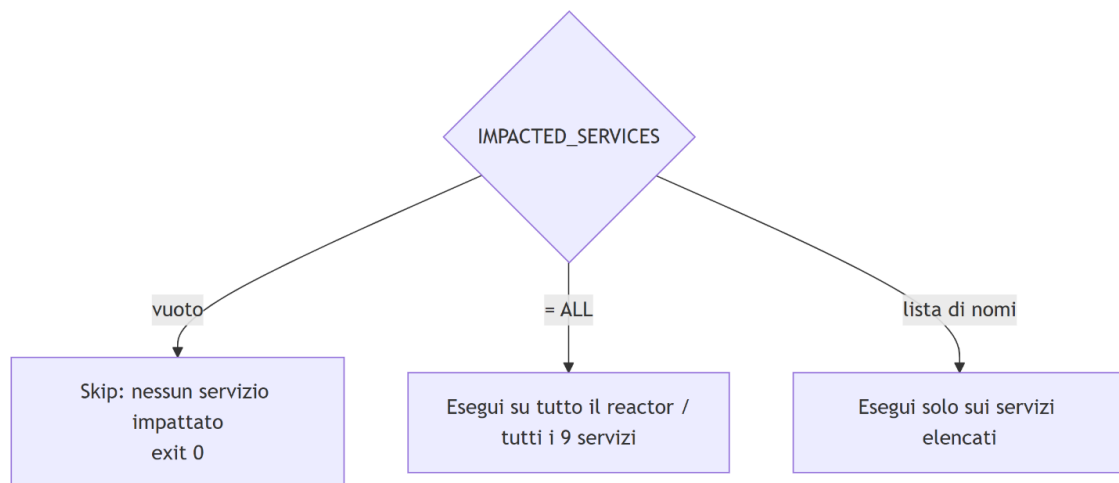


Figura 3.4: Pattern a tre rami applicato da ogni job dipendente da `IMPACTED_SERVICES`: vuoto, `ALL` o lista di servizi.

3.4 Implementazione della soluzione

3.4.1 Analisi delle modifiche (impact detection)

Il cuore dell'intera iniziativa è il job `detect-impact`, eseguito nello stage `init`, riportato di seguito nella sua versione più recente:

```

detect-impact:
  stage: init
  image: alpine:latest
  before_script:
    - apk add --no-cache git
  script:
    - echo "=== Impacted Microservices Analysis ==="
    # CRITERIA 1
    - |
  
```

```

if [ -n "$CI_MERGE_REQUEST_TARGET_BRANCH_NAME" ]; then
TARGET_BRANCH=$CI_MERGE_REQUEST_TARGET_BRANCH_NAME
git fetch origin $TARGET_BRANCH
CHANGED_FILES=$(git diff --name-only origin/$TARGET_BRANCH...HEAD)
else
CHANGED_FILES=$(git diff --name-only $CI_COMMIT_BEFORE_SHA ↔
    $CI_COMMIT_SHA)
fi
# CRITERIA 3
- |
if echo "$CHANGED_FILES" | grep -q -E "(opto-core/|pom\.xml)"; then
IMPACTED_SERVICES="ALL"
else
if echo "$CHANGED_FILES" | grep -q "^project-consumer-boot/"; then ↔
    IMPACTED_SERVICES="$IMPACTED_SERVICES consumer"; fi
#... una condizione analoga per ciascuno dei 9 servizi
fi
- CLEAN_SERVICES=$(echo $IMPACTED_SERVICES | tr ' ' '\n' | sort -u | tr↔
    '\n' ' ' | sed 's/ *$//')
- echo "IMPACTED_SERVICES=$CLEAN_SERVICES" > build.env
artifacts:
reports:
dotenv: build.env
only:
refs: [develop, main, merge_requests]

```

Alcuni aspetti tecnici meritano un commento esplicito:

- **Doppia sorgente del diff:** su pipeline di Merge Request, il confronto avviene con il branch target (`origin/$TARGET_BRANCH...HEAD` — la notazione a tre punti in Git calcola il diff rispetto al *merge-base*, non un diff diretto fra i due riferimenti); su push diretto, il confronto avviene fra `$CI_COMMIT_BEFORE_SHA` e `$CI_COMMIT_SHA`, valori forniti automaticamente da GitLab CI.
- **Mappatura file:** ogni servizio è associato a una condizione `grep -q "^project-<servizio>-boot/"`. È un approccio mantenuto interamente a mano: l'aggiunta di un decimo micro-servizio richiederebbe di ricordarsi di aggiungere la riga corrispondente, un limite approfondito nel Capitolo 5.
- **Regola di fallback ALL:** qualunque modifica a `opto-core/` o al `pom.xml` radice, gli unici due elementi condivisi da tutti i moduli, marca *tutti* i nove microservizi come impattati. È una scelta conservativa e corretta dal punto di vista della sicurezza del rilascio, al costo di una selettività nulla ogniqualevolta il codice condiviso venga toccato, anche marginalmente.
- **Validazione empirica dal vivo:** la fase iniziale del lavoro documenta la verifica pratica dei tre scenari: singolo servizio, più servizi, componente globale, tramite

commit creati e revertati poi appositamente. Trattandosi di una logica interna alla configurazione stessa della pipeline, GitLab CI non offre un ambiente isolato in cui testarla prima di eseguirla realmente: la validazione non è quindi attualmente replicabile in locale, e la scelta di validare direttamente in pipeline reale, con modifiche temporanee e immediatamente revertite, è una metodologia pragmatica per questo vincolo, discussa più diffusamente nel Capitolo 4.

L'introduzione di una correzione di un difetto introdotto in una fase successiva (§3.4.2): un esempio ripreso nel Capitolo 4 come evidenza di un ciclo di sviluppo realmente iterativo, di come l'introduzione della selettività abbia richiesto più di un passaggio per essere corretta in tutti i suoi rami.

Il gate di rifiuto multi-servizio (reject-multi-service-impact)

*Il job è stato **integrato nel branch principale** ed è oggi attivo sulle Merge Request del repository.*

L'ultimo intervento del percorso estende detect-impact in una direzione concettualmente nuova rispetto ai job precedenti: non decide più *cosa* costruire, ma **se la pipeline può proseguire**, applicando come regola di processo il principio "una Merge Request modifica un solo microservizio" — principio che le sezioni precedenti (in particolare il vincolo 4 di §3.2 e il refactoring di disaccoppiamento discusso in §2.1) avevano reso tecnicamente possibile, ma non ancora *imposto*.

```
reject-multi-service-impact:
  stage: init
  image: alpine:latest
  needs:
    - job: detect-impact
  artifacts: true
  script:
    - 'echo "Impacted services: '$IMPACTED_SERVICES''"'
    - |
      # ALL = deliberate shared-code change (opto-core/, root pom.xml) -> ↵
      allowed
      if [ -z "$IMPACTED_SERVICES" ] || [ "$IMPACTED_SERVICES" = "ALL" ]; ↵
      then
        echo "Impact within an allowed scope (none / shared / single). OK."
        exit 0
      fi
    - |
      COUNT=$(echo "$IMPACTED_SERVICES" | wc -w)
      if [ "$COUNT" -gt 1 ]; then
        echo "Pipeline rejected: Multiple microservices impacted (↵
          $IMPACTED_SERVICES)"
```

```

exit 1
fi
echo "Single microservice impacted: $IMPACTED_SERVICES. OK."
only:
refs:
- merge_requests

```

Il job riusa lo stesso pattern a tre rami descritto in §3.3, ma con una semantica in parte invertita rispetto agli altri consumer di IMPACTED_SERVICES: i rami "vuoto" e "ALL" sono **entrambi accettati** (nessun impatto, oppure un impatto condiviso e deliberato su opto-core/pom.xml), mentre il ramo "selettivo" viene accettato solo se un **singolo** servizio è coinvolto — se ne sono coinvolti due o più, la pipeline fallisce esplicitamente (exit 1). Da notare inoltre che, a differenza di detect-impact (eseguito anche su develop e main), questo job gira **solo** su pipeline di Merge Request (only: refs: [merge_requests]), coerentemente con la sua funzione di controllo in fase di revisione, prima che la modifica venga integrata.

La validazione di questo job ha seguito la stessa metodologia "usa e getta" già descritta per detect-impact, con tre scenari reali (singolo servizio, due servizi, modifica condivisa) — la tabella completa degli scenari, con relativi commit, è riportata nel Capitolo 4, §4.2.1.

3.4.2 Build incrementale

Questa sezione descrive due passaggi principali: l'introduzione della logica di build selettiva e una correzione successiva al ramo di gestione del caso in cui tutti i servizi risultano impattati (ALL).

Il diff che introduce la build selettiva in build-maven-project (semplificato):

```

build-maven-project:
script:
- - mvn package $MAVEN_OPTS --settings.m2/settings.xml
+ - |
+ if [ -z "$IMPACTED_SERVICES" ]; then
+ exit 0
+ fi
+ - |
+ MAVEN_PROJECTS=""
+ for service in $IMPACTED_SERVICES; do
+ MAVEN_PROJECTS="${MAVEN_PROJECTS}project-${service}-boot,"
+ done
+ - mvn package $MAVEN_OPTS -DskipTests=true -pl "$MAVEN_PROJECTS" -am -<
  T 1C --settings.m2/settings.xml
+ artifacts:

```

```
+ paths: ["*/target/*.jar"]
+ expire_in: 1 hour
```

La prima versione di questa logica conteneva un difetto, corretto poi ("fix failing maven project build when all services are impacted"): il calcolo di MAVEN_PROJECTS avveniva sempre a partire da \$IMPACTED_SERVICES, ma quando quest'ultima valeva la stringa letterale ALL (e non un elenco di nomi di servizio), il ciclo `for service in $IMPACTED_SERVICES` produceva l'identificatore inesistente `project-ALL-boot`, causando il fallimento della build. La correzione introduce un ramo effettivo per il caso ALL, che esegue `mvn package` senza l'opzione `-pl` (quindi sull'intero reactor):

```
+ if [ "$IMPACTED_SERVICES" = "ALL" ]; then
+ mvn package $MAVEN_OPTS -T 1C --settings.m2/settings.xml
+ else
  MAVEN_PROJECTS=""
  for service in $IMPACTED_SERVICES; do...
+ mvn package $MAVEN_OPTS -pl "$MAVEN_PROJECTS" -am -T 1C --settings.m2/↵
  settings.xml
+ fi
```

Da un punto di vista tecnico, l'opzione `-pl` (*projects list*) del Maven Reactor limita la build ai moduli indicati; `-am` (*also make*) aggiunge automaticamente al reactor le dipendenze upstream di tali moduli, garantendo che i moduli condivisi da cui dipende un servizio impattato vengano comunque (ri)compilati; `-T 1C` attiva l'esecuzione parallela della build, con un thread per core disponibile sul runner. La Figura 3.5 confronta la build completa con quella selettiva a fronte della stessa modifica.

3.4.3 Automazione del deploy

Questa sezione tratta due interventi distinti ma strettamente collegati: l'introduzione dell'autenticazione OIDC verso AWS (prerequisito di sicurezza) e il job `deploy-prod` che automatizza il rilascio in produzione.

Autenticazione OIDC.

```
build-and-push-images:
  image: maven:3-amazoncorretto-21-alpine
+ id_tokens:
+ AWS_ID_TOKEN:
+ aud: https://sts.amazonaws.com
  before_script:
  - apk add --no-cache python3 py3-pip bash libxml2-utils aws-cli docker
  - chmod +x builder.sh
```

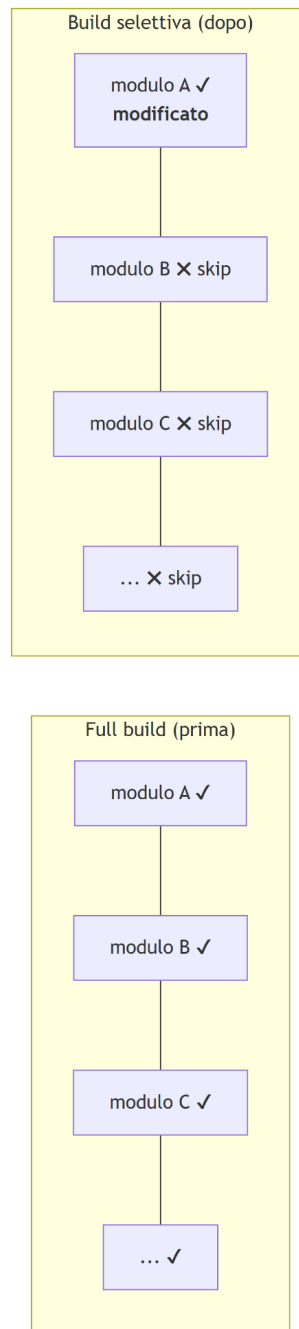


Figura 3.5: Full build (prima) vs. build selettiva (dopo) a fronte della modifica di un solo modulo.

```

+ - echo "${AWS_ID_TOKEN}" > "${CI_PROJECT_DIR}/.aws_web_identity_token"
+ - export AWS_WEB_IDENTITY_TOKEN_FILE="${CI_PROJECT_DIR}/.aws_web_identity_token"
+ - export AWS_ROLE_SESSION_NAME="gitlab-${CI_PROJECT_NAME}-${CI_PIPELINE_ID}"
+ - aws sts get-caller-identity

```

Come introdotto concettualmente nel Capitolo 2, §2.3, GitLab genera per ogni esecuzione di pipeline un token JWT firmato (AWS_ID_TOKEN), con aud: `https://sts.amazonaws.com`; l'AWS CLI, trovando la variabile `AWS_WEB_IDENTITY_TOKEN_FILE` valorizzata in ambiente, lo scambia automaticamente con AWS STS per ottenere credenziali temporanee attraverso il meccanismo `AssumeRoleWithWebIdentity` — a condizione che, sul lato AWS, sia configurato un IAM OIDC Identity Provider che si fidi dell'issuer GitLab. Il comando `aws sts get-caller-identity`, eseguito subito dopo, funge da verifica esplicita che l'autenticazione OIDC abbia effettivamente avuto successo — una buona pratica di fail-fast in pipeline. La sequenza completa di autenticazione è illustrata in Figura 3.6.

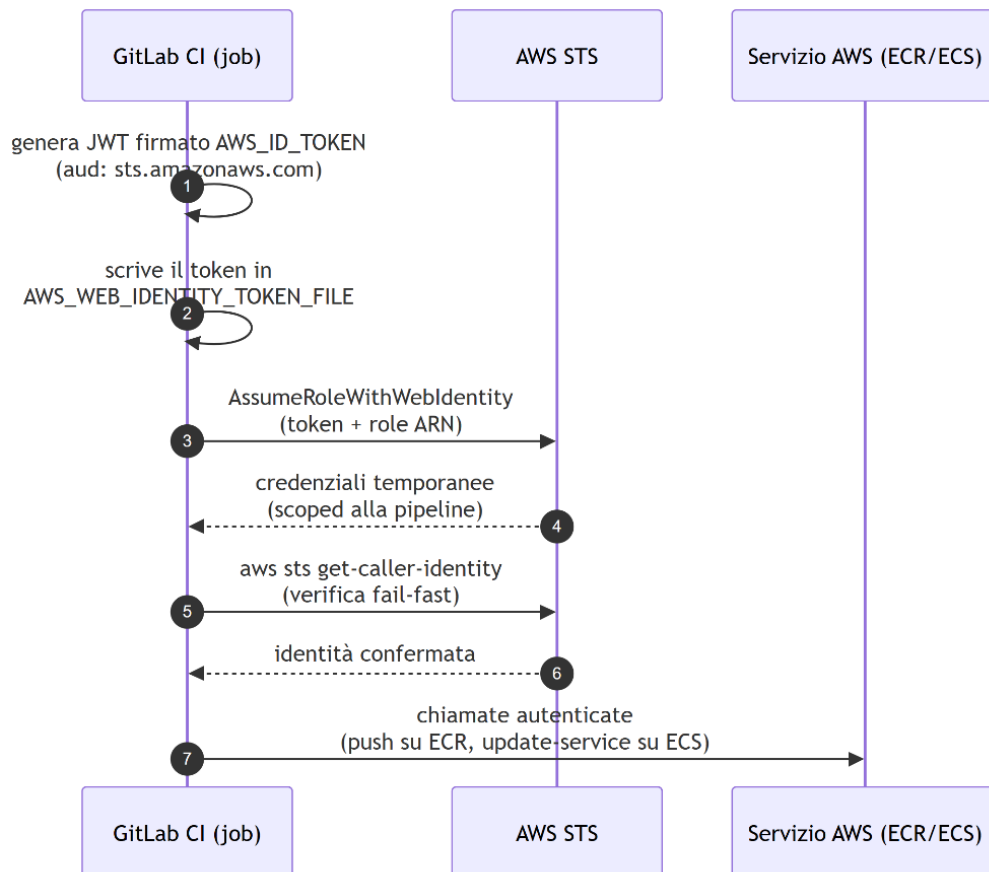


Figura 3.6: Sequenza di autenticazione OIDC verso AWS: dal JWT firmato da GitLab CI alle credenziali STS temporanee.

Il job deploy-prod.

```

deploy-prod:
  stage: deploy-prod
  needs:
  - job: detect-impact
  artifacts: true
  - job: build-and-push-images
  id_tokens:
    AWS_ID_TOKEN: { aud: https://sts.amazonaws.com }
  before_script:
  - apk add --no-cache aws-cli jq bash
  - echo "${AWS_ID_TOKEN}" > "${CI_PROJECT_DIR}/.aws_web_identity_token"
  - export AWS_WEB_IDENTITY_TOKEN_FILE="${CI_PROJECT_DIR}/.↵
    aws_web_identity_token"
  - aws sts get-caller-identity
  script:
  - IMAGE_TAG=${4:-${CI_COMMIT_REF_SLUG}-${CI_COMMIT_SHORT_SHA}}
  - |
  if [ "$IMPACTED_SERVICES" = "ALL" ]; then
    SERVICES_TO_DEPLOY="consumer gateway migration registry scheduler web ↵
      auth measurement analytics"
  else
    SERVICES_TO_DEPLOY="$IMPACTED_SERVICES"
  fi
  - |
  for svc in $SERVICES_TO_DEPLOY; do
    TASK_DEF=$(aws ecs describe-task-definition --task-definition "optovera↵
      -${svc}-prod" --query taskDefinition)
    NEW_TASK_DEF=$(echo "$TASK_DEF" | jq \
      --arg IMAGE "${AWS_ECR_REGISTRY}/optovera-${svc}:${IMAGE_TAG}" \
      '.containerDefinitions[0].image = $IMAGE
    | del(.taskDefinitionArn,.revision,.status,.requiresAttributes,.↵
      compatibilities,.registeredAt,.registeredBy)')
    NEW_TASK_DEF_ARN=$(aws ecs register-task-definition --cli-input-json "↵
      $NEW_TASK_DEF" --query 'taskDefinition.taskDefinitionArn' --output ↵
      text)
    aws ecs update-service --cluster "optovera-${svc}-prod" --service "↵
      optovera-${svc}-prod" --task-definition "$NEW_TASK_DEF_ARN"
  done

```

Il job implementa il pattern, canonico per Amazon ECS, delle **task definition immutabili**: poiché una task definition esistente non può essere modificata direttamente, il job ne clona la revisione corrente (`describe-task-definition`), ne sostituisce con `jq` esclusivamente il campo `image`, rimuove i campi calcolati automaticamente da AWS non accettati in input

(taskDefinitionArn, revision, status, tra gli altri — la loro presenza farebbe fallire register-task-definition), e registra così una **nuova revisione**. Il service ECS viene infine aggiornato con update-service -task-definition <nuovo ARN>, che avvia autonomamente un *rolling deployment*, sostituendo gradualmente i task in esecuzione con la nuova revisione. La Figura 3.7 riassume il meccanismo di aggiornamento tramite revisione immutabile della task definition.

Un fix successivo corregge un errore di naming introdotto nella versione precedente del job, con un impatto diretto sulla correttezza del deploy: senza questa correzione, l'aggiornamento avrebbe puntato a un cluster ECS condiviso inesistente invece che al cluster dedicato del singolo servizio.

```
- aws ecs update-service --cluster optovera-prod --service "optovera-#{←
  svc}-prod"...
+ aws ecs update-service --cluster "optovera-#{svc}-prod" --service "←
  optovera-#{svc}-prod"...
```

La versione iniziale assumeva un **unico cluster ECS condiviso** (optovera-prod) per tutti i servizi; il fix rivela che l'infrastruttura reale prevede invece **un cluster ECS dedicato per ciascun servizio** (optovera-<servizio>-prod). È un esempio concreto, ripreso nel Capitolo 5, di un accoppiamento implicito che sopravvive anche dopo il disaccoppiamento del ciclo di rilascio applicativo da Terraform: la pipeline applicativa deve comunque conoscere con esattezza il naming delle risorse infrastrutturali definite altrove.

3.4.4 Containerizzazione dei microservizi

Questa sezione ripercorre l'introduzione della funzione build_selective, la successiva disabilitazione temporanea della build immagini sulle pipeline di Merge Request, e l'autenticazione OIDC già discussa in §3.4.3.

Lo script bash builder.sh, invocato dal job build-and-push-images, definisce funzioni riutilizzabili per la costruzione delle immagini Docker:

```
function build_image() {
  local PROJECT_NAME=${1}; local PROFILE=${3}
  local IMAGE_TAG=${4:-${CI_COMMIT_REF_SLUG:-...}-${CI_COMMIT_SHORT_SHA←
    :-...}}
  docker build \
    --build-arg "PROJECT_NAME=${PROJECT_NAME}" \
    --build-arg "PROJECT_VERSION=${PROJECT_VERSION}" \
    --build-arg "PROFILE=${PROFILE}" \
    --tag optovera/project-${PROJECT_NAME}:${IMAGE_TAG} \
    --provenance=false.
}
```

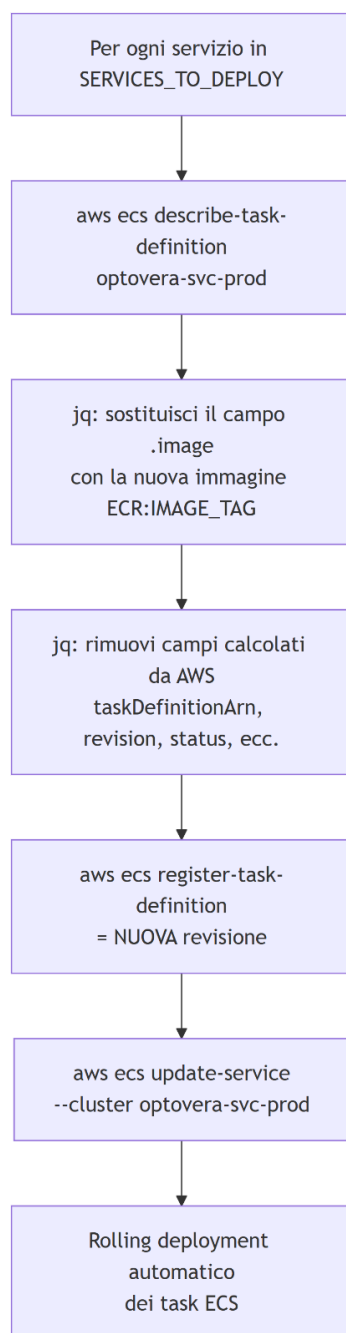


Figura 3.7: Meccanismo di deploy su Amazon ECS tramite revisione immutabile della task definition.

```
function build_selective() {
  local PROFILE=${1}; local IMPACTED_SERVICES=${2}
  local ALL_SERVICES=(project-registry-boot project-gateway-boot...)
  for service in "${ALL_SERVICES[@]"; do
    if echo "${IMPACTED_SERVICES}" | grep -qw "${service}" ||...; then
      "build_${service//-/}_)" "${PROFILE}" "$@" # dispatch dinamico
    else
      echo ">> [SKIP] Unaltered dependency: ${service}"
    fi
  done
}
```

Il punto di interesse tecnico principale è il **dispatch dinamico di funzioni bash** in `build_selective()`: invece di un blocco `if/elif` esplicito per ciascun servizio, la funzione costruisce dinamicamente il nome della funzione da invocare (`build_${service//-/}_)` e la chiama indirettamente solo per i servizi effettivamente impattati. Il pattern è riutilizzabile e testabile anche da riga di comando (`./builder.sh build_selective prod "web auth"`), indipendentemente da GitLab CI – un buon esempio di script pensato per essere eseguibile anche in locale a scopo di debug, come suggerito dal messaggio d’uso finale dello script stesso. Il dispatch dinamico interno a `build_selective` è schematizzato in Figura 3.8.

Il job `build-and-push-images` (stage `images`), nella parte che invoca lo script e gestisce la pubblicazione su ECR:

```
script:
- |
  if [ "$IMPACTED_SERVICES" = "ALL" ]; then
    ./builder.sh build_all prod
  else
    ./builder.sh build_selective prod "$IMPACTED_SERVICES"
  fi
- |
  process_image() {
    local svc=$1; local ecr_url=$2
    if [ "$IMPACTED_SERVICES" = "ALL" ] || echo "$IMPACTED_SERVICES" | grep ←
      -qw "$svc"; then
      docker tag optovera/project-${svc}-boot:$IMAGE_TAG $ecr_url:$IMAGE_TAG
      docker push $ecr_url:$IMAGE_TAG
    fi
  }
  process_image "consumer" $ECR_CONSUMER
  #... ripetuto per i 9 servizi
```

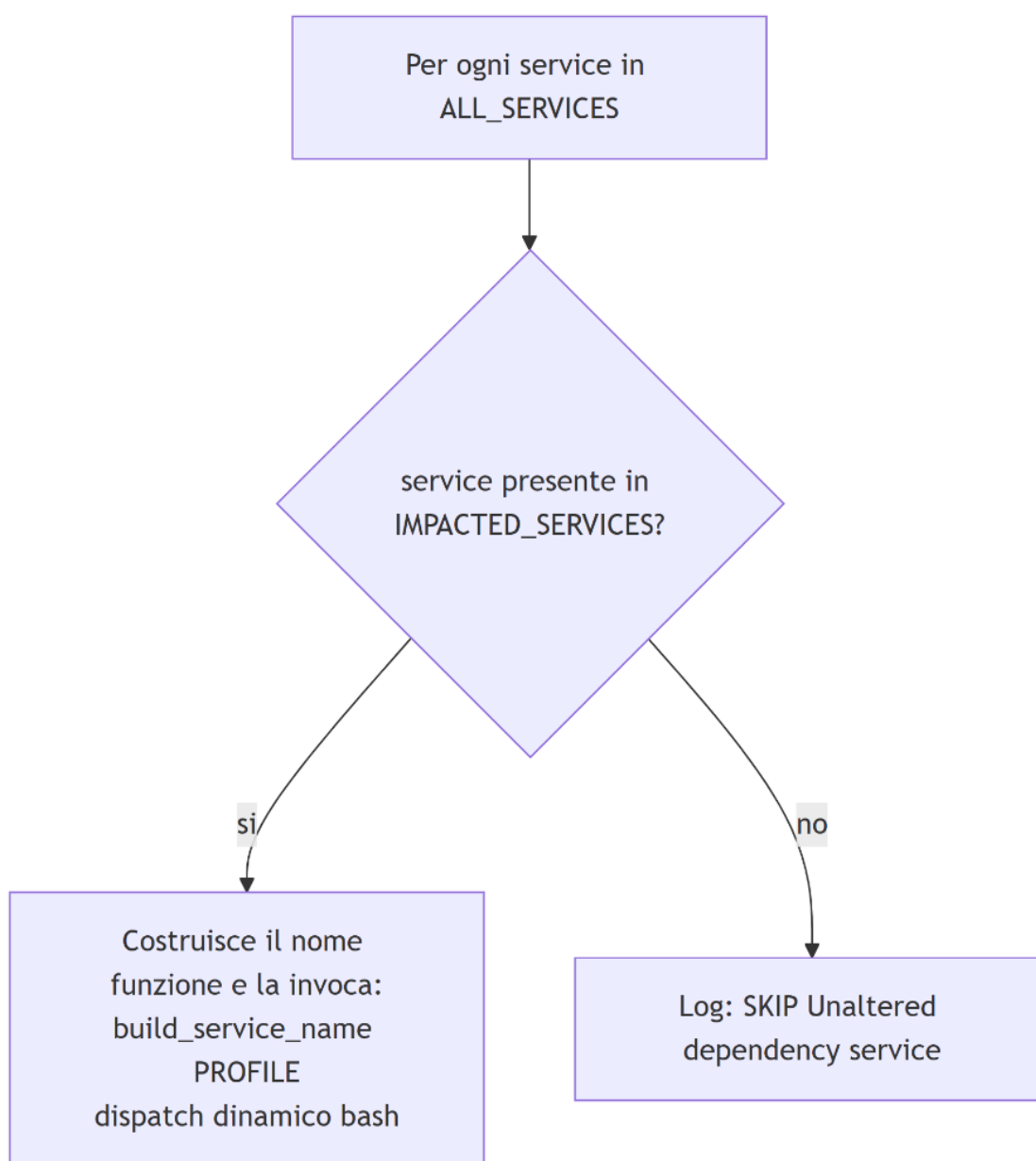


Figura 3.8: Dispatch dinamico in build_selective: invocazione della funzione di build per ogni servizio impattato.

È utile segnalare la **ridondanza fra `builder.sh`** (che decide cosa costruire) e la **funzione `process_image inline`** (che decide cosa taggare e pubblicare): entrambe implementano lo stesso controllo di appartenenza a `IMPACTED_SERVICES`, ma in punti diversi del codice — una condizione di disallineamento potenziale se una delle due venisse modificata senza aggiornare anche l'altra, ripresa nel Capitolo 5 come spunto di miglioramento.

L'esecuzione di questo job è stata temporaneamente disabilitata sulle pipeline di Merge Request, mantenendolo solo su `main`: una scelta di costo/rischio esplicita — non pubblicare immagini Docker per ogni MR, ma solo per i commit effettivamente integrati.

3.4.5 Gestione degli artifact

Nel progetto convivono due meccanismi GitLab CI concettualmente distinti per il passaggio di informazioni fra job:

- **Artifact "normali"** (`artifacts.paths`): utilizzati da `mvninit` (che produce `.m2/settings.xml`, con TTL di un'ora) e da `build-maven-project` (che produce `*/target/*.jar`, stesso TTL), per trasferire file fisici fra job e stage differenti.
- **Artifact "dotenv"** (`artifacts.reports.dotenv`): utilizzato esclusivamente da `detect-impact` per esportare `IMPACTED_SERVICES` come vera e propria **variabile d'ambiente**, automaticamente disponibile in ogni job a valle che dichiarare `needs: [{job: detect-impact, artifacts: true}]`. È un meccanismo nativo di GitLab CI, che non richiede parsing manuale di un file di output, introdotto contestualmente alla build selettiva.

La distinzione concettuale fra i due meccanismi è utile da esplicitare: gli artifact "paths" trasportano *contenuti* (i JAR compilati), mentre l'artifact "dotenv" trasporta *stato di decisione* (quali servizi sono stati giudicati impattati), due categorie di informazione differenti che attraversano gli stage della stessa pipeline con meccanismi diversi.

A un livello di astrazione superiore, anche le **immagini Docker** pubblicate su Amazon ECR possono essere lette come un artifact: taggate con `${CI_COMMIT_REF_SLUG}-${CI_COMMIT_SHORT_SHA}` e pubblicate su un repository per servizio, costituiscono l'artifact finale che attraversa il confine fra CI (build) e CD (il job `deploy-prod`, che recupera esattamente lo stesso `IMAGE_TAG` per identificare l'immagine corretta da distribuire su ECS).

4

CAPITOLO 4

Sperimentazione e risultati

4.1 Scenario sperimentale

Questo capitolo affronterà, a differenza di un approccio più tradizionale basato su ambienti di test isolati, una validazione condotta sulla pipeline presa in esame: la configurazione stessa della CI/CD non si presta a un ambiente di test separato, per cui ogni comportamento atteso è stato verificato introducendo modifiche reali, temporanee e successivamente revertate, osservandone l'effetto concreto sull'esecuzione della pipeline stessa. Lo scenario copre due meccanismi distinti, entrambi direttamente rilevanti per l'obiettivo primario, la produzione:

- La correttezza dell'**analisi d'impatto** (detect-impact), cioè la capacità della pipeline di individuare correttamente quali microservizi sono coinvolti da una modifica; presupposto di ogni build, immagine e deploy selettivo a valle.
- La correttezza del **gate di rifiuto multi-servizio** (reject-multi-service-impact), cioè la capacità della pipeline di impedire, prima ancora del merge, che una modifica multi-servizio comprometta la disciplina che rende possibile un deploy in produzione realmente indipendente per servizio.

4.2 Flusso operativo della pipeline

4.2.1 Analisi delle modifiche

Durante lo sviluppo, gli scenari fondamentali della logica di impact detection sono stati validati singolarmente. La Tabella 4.1 riporta, per ciascuno scenario, la modifica di prova introdotta e successivamente revertata, e i servizi che detect-impact rileva come impattati.

Ciascuno scenario conferma un ramo distinto della logica descritta nel Capitolo 3, §3.4.1: modifica confinata → un solo servizio in IMPACTED_SERVICES; modifica distribuita su

Scenario	Servizi rilevati
Modifica a un singolo servizio	consumer
Modifica a un singolo servizio	web
Modifica a più servizi contemporaneamente	auth + gateway
Modifica a componente globale (opto-core/)	tutti i 9 servizi (ALL)
Modifica a componente globale (pom.xml radice)	tutti i 9 servizi (ALL)

Tabella 4.1: Scenari di verifica dell'analisi d'impatto: modifica introdotta e microservizi rilevati da detect-impact.

più servizi → più nomi nell'elenco; modifica a codice condiviso (opto-core/ o pom.xml radice) → fallback ALL.

Un dettaglio merita di essere esplicitato, perché renda la validazione più leggibile e più difendibile: **la rilevazione funziona anche sulle cancellazioni**. Lo scenario multi-servizio in questo caso non aggiunge codice, ma **elimina** un file di test in project-auth-boot e una classe in project-gateway-boot. Poiché detect-impact si basa su git diff -name-only, che elenca i file cancellati esattamente come quelli modificati o aggiunti, entrambi i servizi vengono correttamente marcati come impattati: un'evidenza che la mappatura percorso→servizio è robusta rispetto al tipo di modifica, non solo alle aggiunte.

Validazione del gate reject-multi-service-impact

Il gate è oggi attivo sulle Merge Request del repository.

Il gate introdotto nell'ultima fase del lavoro è stato validato con la stessa metodologia, su tre scenari che non misurano quale servizio viene costruito, ma se la pipeline prosegue o viene rifiutata: una modifica a un singolo servizio lascia la pipeline OK; una modifica contemporanea a due servizi fa terminare la pipeline con un rifiuto esplicito (exit 1); una modifica condivisa al pom.xml radice, che coinvolge tutti e nove i servizi, lascia nuovamente la pipeline OK.

A differenza degli scenari precedenti per detect-impact (che modificavano file applicativi reali), qui i primi due scenari usano un file marcatore dedicato e innocuo, uno per servizio coinvolto, invece di modificare codice realmente impattante sul progetto in corso: un affinamento della tecnica di validazione, più isolato e meno rischioso. Solo il terzo scenario tocca effettivamente pom.xml, perché deve simulare autenticamente una modifica condivisa reale: non esiste un "marcatore" equivalente per il caso ALL, dato che la regola di fallback in detect-impact è ancorata esplicitamente ai percorsi opto-core/ e pom.xml (Capitolo 3, §3.4.1).

Questo scenario, la rejection del caso "due servizi", è la verifica più direttamente collegata all'obiettivo primario dichiarato in questa tesi: è la prova empirica che la pipeline impedisce,

prima del merge, una condizione che comprometterebbe la possibilità di un deploy in produzione realmente indipendente per servizio.

4.2.2 Build e deploy

La build selettiva viene esercitata dagli stessi tre scenari di §4.2.1: nel caso di modifica a un singolo servizio, il job `build-maven-project` (Cap. 3, §3.4.2) esegue `mvn package -pl project-<servizio>-boot -am -T 1C`, producendo un solo artifact JAR invece dei nove che venivano prodotti dalla pipeline monolitica di partenza (Cap. 1, §1.2). Il log del job riporta esplicitamente i moduli inclusi nella build (`Executing selective build for modules:...`), evidenza diretta e verificabile del comportamento selettivo.

Un caso limite da presentare esplicitamente: la build selettiva, tramite l'opzione `-am`, ricompila anche i moduli condivisi (ad es. `opto-core`) se dichiarati come dipendenza del servizio impattato. "Selettivo", in questo contesto, non significa "isolato al 100%", ma quantomeno coinvolgere il minimo necessario dato il grafo delle dipendenze Maven, una precisazione già discussa in Cap. 3, §3.4.2, che va ribadita qui in chiave sperimentale.

Sul fronte del deploy, poiché la produzione è l'obiettivo primario di questa tesi, questa fase è la più rilevante ai fini della valutazione complessiva del lavoro. Ad è oggi possibile documentare un **rilascio reale in produzione**, si tratta della prova sperimentale più diretta: la pipeline ha aggiornato autonomamente servizi reali su Amazon ECS, in modo selettivo.

Il rilascio in produzione (traccia reale)

Le Merge Request che portavano i due branch finali sono state rilasciate fino a main. La cronologia dei merge su main è riportata, in ordine cronologico, come segue in Tabella 4.2:

Ora (CEST)	Contenuto portato in main
09:56	Modifiche applicative a <code>project-measurement-boot</code> e <code>project-web-boot</code>
10:17	Job <code>deploy-prod</code> in <code>.gitlab-ci.yml</code>
11:05	Job <code>reject-multi-service-impact</code>

Tabella 4.2: Cronologia dei merge confluiti in main nel rilascio reale in produzione.

L'aspetto tecnicamente rilevante è **come** `detect-impact` ha interpretato questo insieme di modifiche. Su una pipeline di main (non di Merge Request), il job calcola il diff fra `$CI_COMMIT_BEFORE_SHA` e `$CI_COMMIT_SHA`, cioè fra lo stato precedente di main e il commit di testa del push. Poiché i tre merge sono confluiti in main in un unico avanzamento del ramo, il diff complessivo, calcolato dal precedente tip di main fino al nuovo commit

di testa, conteneva modifiche a `project-web-boot/`, a `project-measurement-boot/` e a `.gitlab-ci.yml`, ma **nessuna** modifica a `opto-core/` o al `pom.xml` radice. Di conseguenza `detect-impact` non ha attivato il fallback ALL, e ha prodotto:

```
IMPACTED_SERVICES=measurement web
```

La traccia di esecuzione end-to-end che ne è seguita è dunque quella di un **deploy selettivo a due servizi su nove**:

- **Push su main** contenente i tre merge commit.
- **Stage init**: `detect-impact` calcola `CHANGED_FILES` sull'intero arco del push, esclude il fallback ALL (nessun percorso condiviso toccato), rileva `^project-web-boot/` e `^project-measurement-boot/`, produce `IMPACTED_SERVICES="measurement web"` e lo esporta come artifact `dotenv (build.env)`.
- **Stage build**: `build-maven-project` legge le due voci ed esegue la build selettiva dei soli moduli `project-measurement-boot` e `project-web-boot` (più le rispettive dipendenze upstream via `-am`).
- **Stage images**: `build-and-push-images` costruisce e pubblica su ECR le sole immagini di `measurement` e `web`, saltando gli altri sette servizi (`log » [SKIP] Unaltered dependency: ...`).
- **Stage deploy-prod**: il job itera su `SERVICES_TO_DEPLOY="measurementweb"`, registra una nuova revisione della task definition ECS per `optovera-measurement-prod` e `optovera-web-prod` e aggiorna i due service corrispondenti con `aws ecs update-service`. Gli altri sette servizi (`auth`, `analytics`, `gateway`, `registry`, `consumer`, `migration`, `scheduler`) restano alla revisione precedente, senza downtime né redeploy superfluo.

Questo è, concretamente, il comportamento che l'intero lavoro si prefiggeva di rendere possibile: un rilascio in produzione **automatico** (nessun intervento manuale sulla console AWS), **indipendente per servizio** (due service aggiornati, sette invariati) e **innescato dalla sola integrazione del codice** in `main`.

Nota metodologica sulla granularità. Va segnalato che il deploy ha coinvolto due servizi in un'unica esecuzione non perché una singola modifica ne toccasse due, ma perché il push su `main` conteneva più Merge Request accorpate, accumulate nel tempo intercorso senza un rilascio in produzione: uno scenario non specificamente gestito da questo lavoro. Su `main`, infatti, l'unità di selettività effettiva è il **push**, non la singola MR, e il gate `reject-multi-service-impact` — che gira *only: merge_requests* — non interviene su `main`. Questo non contraddice l'indipendenza del deploy (i sette servizi non toccati restano invariati), ma individua un limite reale della soluzione, discusso in dettaglio nel Capitolo 5, §5.1.2.

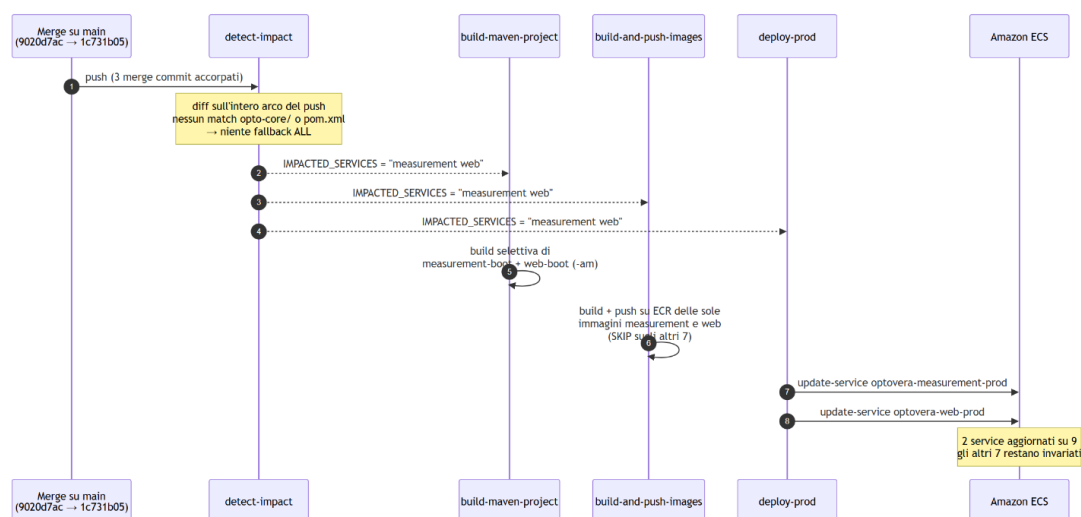


Figura 4.1: Traccia end-to-end del rilascio reale in produzione: dal push su main al deploy selettivo su ECS (measurement, web).

4.3 Valutazione delle prestazioni

4.3.1 Riduzione dei tempi

I dati quantitativi di durata delle pipeline sono oggi disponibili grazie alla cronologia delle esecuzioni reali in GitLab CI/CD. Il confronto è condotto su entrambi gli ambienti toccati dal lavoro di selettività: la pipeline di staging (branch develop, job deploy-stage) e la pipeline di produzione (branch main, job deploy-prod), mettendo a confronto un'esecuzione precedente all'introduzione della selettività con esecuzioni successive, sia in assenza sia in presenza di un microservizio effettivamente impattato.

Nota metodologica. I valori riportati in questo paragrafo provengono da una singola esecuzione osservata per ciascuna condizione: non sono medie su ripetizioni e non è quindi possibile stimarne la dispersione. Il confronto, inoltre, non è condotto a parità di struttura della pipeline: l'esecuzione di partenza comprendeva 5 job, quelle successive 8, e due dei job attuali (build-maven-project e deploy-prod) non esistevano nella configurazione iniziale, per cui le durate complessive vanno lette come confronto fra due configurazioni reali successive, non come misura isolata del solo effetto della selettività. I valori indicati come «< 20 s» in Tabella 4.3 e in Tabella 4.4 sono soglie di lettura del grafico delle esecuzioni GitLab, non misure puntuali.

Pipeline di produzione (main → deploy-prod). Prima dell'introduzione della selettività, un'esecuzione osservata sulla pipeline di main comprendeva 5 job e richiedeva complessivamente 15 minuti e 50 secondi, poiché ogni job operava sempre sull'intero reactor Maven e su tutti i nove servizi: i due più onerosi erano

sonarqube-check (9 minuti e 24 secondi) e build-and-push-images (4 minuti e 56 secondi) (Figura 4.2).

Dopo l'introduzione della selettività, la stessa pipeline è stata osservata in due condizioni. In assenza di microservizi impattati, l'intera esecuzione si completa in 1 minuto e 49 secondi: detect-impact determina in pochi secondi che nessun servizio è coinvolto, e i job di analisi e pubblicazione immagine vedono di fatto azzerato il proprio carico di lavoro (Figura 4.3). In presenza di un singolo microservizio impattato (measurement), l'esecuzione richiede 12 minuti e 40 secondi complessivi, con build-maven-project a 4 minuti e 18 secondi, sonarqube-check a 6 minuti e 11 secondi, build-and-push-images a 23 secondi e deploy-prod a 10 secondi (Figura 4.4).

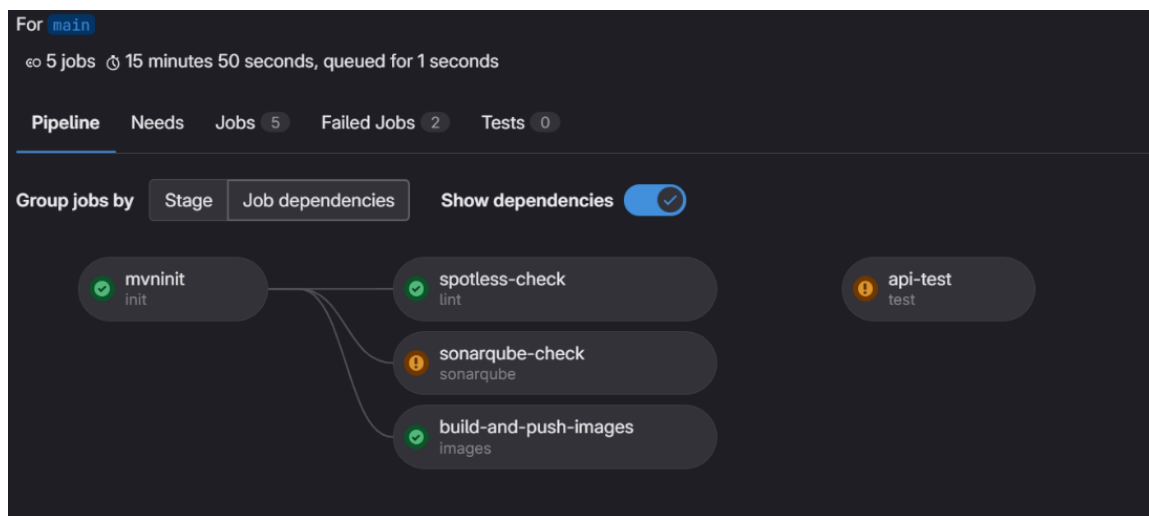


Figura 4.2: Pipeline su main prima dell'introduzione della selettività: 5 job, nessuna selettività, 15 min 50 s complessivi.

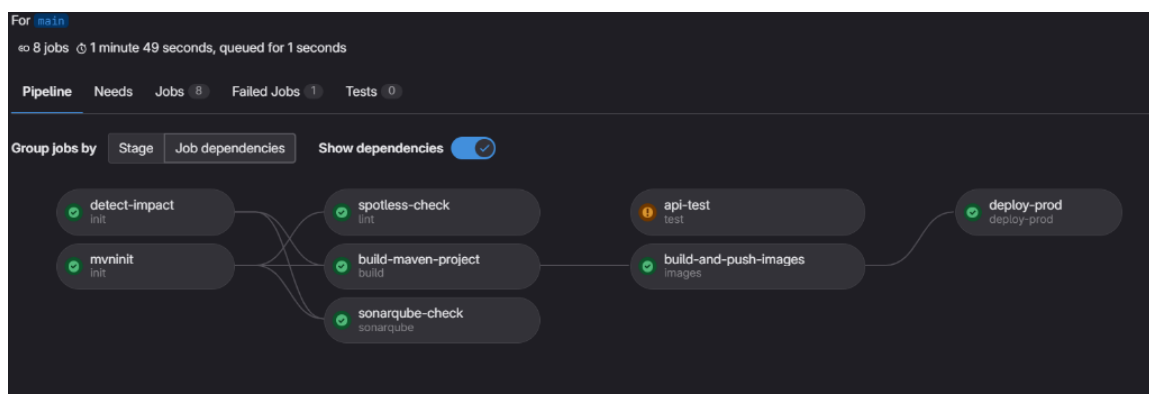


Figura 4.3: Pipeline su main dopo la selettività, nessun microservizio impattato: 8 job, 1 min 49 s complessivi.

Il confronto mostra una riduzione marcata anche nello scenario a singolo servizio impattato, non solo in quello privo di impatto: sonarqube-check passa da 9 minuti e 24 secondi a 6

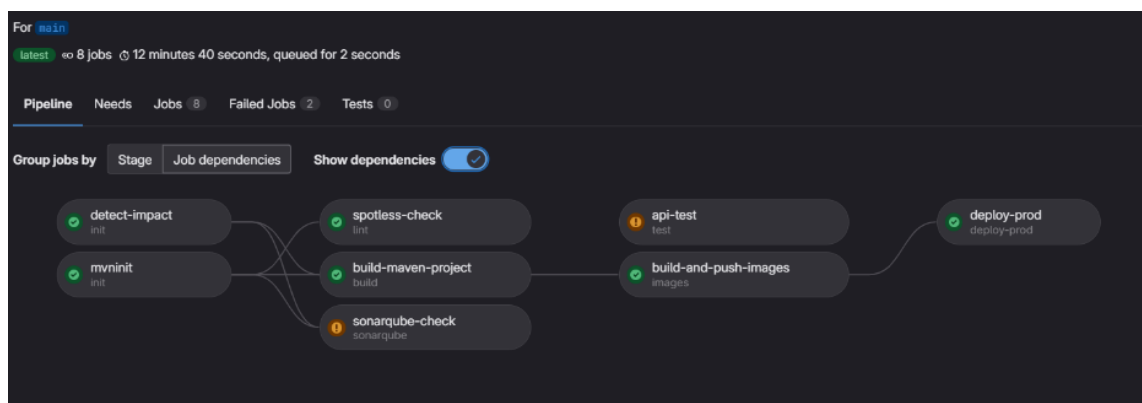


Figura 4.4: Pipeline su main dopo la selettività, un microservizio impattato (measurement): 8 job, 12 min 40 s complessivi.

minuti e 11 secondi (circa -34%) e build-and-push-images da 4 minuti e 56 secondi a 23 secondi (circa -92%), poiché entrambi operano ora su un solo modulo anziché sull'intero reactor. Va però osservato che il guadagno sulla durata complessiva è sensibilmente più contenuto di quello sui singoli job: 12 minuti e 40 secondi contro 15 minuti e 50 secondi, circa il 20% in meno. Il tempo residuo è quasi interamente attribuibile a sonarqube-check, che resta il job più oneroso con 6 minuti e 11 secondi anche quando l'analisi riguarda un solo modulo, perché il costo dell'analisi statica non scala linearmente col numero di moduli analizzati: comprende una quota fissa di inizializzazione, di risoluzione dei plugin e di comunicazione col server SonarQube. La selettività elimina quindi il lavoro ridondante sui moduli non impattati, ma non riduce il costo unitario dell'analisi del servizio effettivamente modificato. Il quadro completo dei job, nelle tre condizioni osservate, è riportato in Tabella 4.3.

Job	Prima	Dopo – nessun impatto	Dopo – 1 servizio impattato
Durata totale pipeline	15 min 50 s	1 min 49 s	12 min 40 s
sonarqube-check	9 min 24 s	< 20 s	6 min 11 s
build-and-push-images	4 min 56 s	< 20 s	23 s
build-maven-project	job non ancora introdotta	< 20 s	4 min 18 s
deploy-prod	job non ancora introdotta	10 s	10 s

Tabella 4.3: Confronto delle durate sulla pipeline di produzione (main).

Pipeline di staging (develop → deploy-stage). Prima dell'introduzione della selettività, un'esecuzione osservata sulla pipeline di develop comprendeva 4 job e richiedeva complessivamente 14 minuti e 35 secondi: il job deploy-stage impiegava 7 minuti e 27 secondi per effettuare il deploy dell'intero backend, avviando i contai-

ner Docker di tutti i microservizi a ogni esecuzione, mentre build-maven-project impiegava 6 minuti e 30 secondi per compilare l'intero reactor (Figura 4.5).

Dopo l'introduzione della selettività, l'overhead del job detect-impact è risultato irrisorio (3 secondi). Con nessun microservizio impattato, l'intera pipeline si completa in 8 minuti e 5 secondi: build-maven-project si riduce a 4 secondi, ma deploy-stage resta a 7 minuti e 15 secondi, sostanzialmente invariato, poiché continua a effettuare il deploy di tutti i microservizi indipendentemente da quali siano stati modificati (Figura 4.6). Con un singolo microservizio impattato (measurement), build-maven-project impiega 4 minuti e 16 secondi.

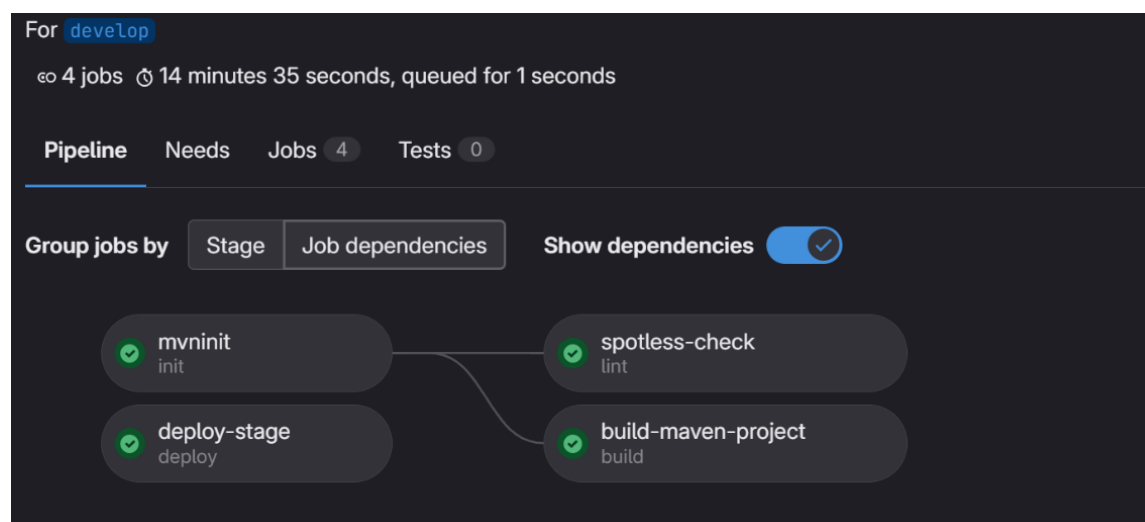


Figura 4.5: Pipeline su develop prima dell'introduzione della selettività: 4 job, 14 min 35 s complessivi.

La riduzione della durata complessiva (-45% circa, da 14 minuti e 35 secondi a 8 minuti e 5 secondi) è quasi interamente attribuibile al collasso di build-maven-project, mentre deploy-stage rimane il costo dominante della pipeline di staging: non essendo stato reso selettivo in questo lavoro, resta un limite riconosciuto, coerente con quanto discusso nel Capitolo 5. Il confronto completo delle durate di staging è riportato in Tabella 4.4.

Job	Prima	Dopo – nessun impatto	Dopo – 1 servizio impattato
Durata totale pipeline	14 min 35 s	8 min 5 s	n/d
deploy-stage	7 min 27 s	7 min 15 s	n/d
build-maven-project	6 min 30 s	4 s	4 min 16 s
detect-impact	job non ancora introdotto	3 s	n/d

Tabella 4.4: Confronto delle durate sulla pipeline di staging (develop).

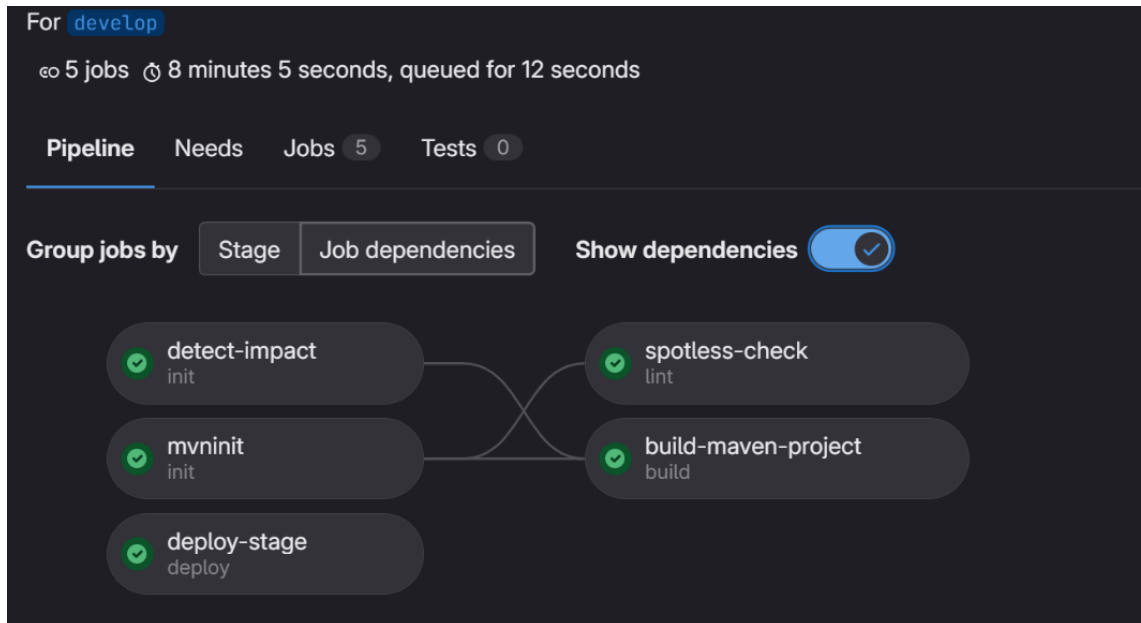


Figura 4.6: Pipeline su develop dopo la selettività, nessun microservizio impattato: 5 job, 8 min 5 s complessivi.

4.3.2 Indipendenza di build e deploy

Come mostrato in §4.2.2, una build selettiva produce solo gli artifact JAR dei servizi impattati (`project-<servizio>-boot/target/*.jar`), non tutti e nove. È un'evidenza verificabile direttamente dagli `artifacts.paths` dichiarati dal job `build-maven-project` e dal contenuto del log (`Executing selective build for modules:...`), senza necessità di dati di temporizzazione. Nel rilascio reale in produzione (§4.2.2), ad esempio, la build ha prodotto i soli artifact di `project-measurement-boot` e `project-web-boot`.

Sul versante del deploy, coerentemente con la priorità dichiarata in apertura di questo capitolo, si colloca l'evidenza più direttamente legata all'obiettivo primario della tesi, e il rilascio in produzione (§4.2.2) ne costituisce la prova empirica: il job `deploy-prod` ha aggiornato **due** ECS service su nove (`measurement`, `web`), lasciando gli altri sette alla revisione precedente. L'evidenza dell'indipendenza non risiede in un accesso diretto all'ambiente AWS — non disponibile nell'ambito di questo lavoro — ma nel **log del job `deploy-prod`** stesso, che è output della pipeline e quindi parte integrante del repository/CI: il job itera esplicitamente sui soli servizi presenti in `SERVICES_TO_DEPLOY`, stampando per ciascuno la riga » `[DEPLOY] Deploying <svc> to ECS with tag <tag>` e il nuovo ARN registrato (» `[DONE] <svc> updated to <arn>`). L'assenza dei sette servizi non impattati da questo elenco è la dimostrazione, verificabile senza credenziali AWS, che nessuna revisione è stata registrata per essi (si veda la traccia end-to-end in Figura 4.1, §4.2.2).

Questo risultato può essere collegato, a titolo di framework di lettura complementare — non come obiettivo primario di questa tesi (si veda Cap. 1, §1.3) — alle metriche DORA introdotte nel Cap. 2, §2.8:

- **Deployment Frequency:** deploy indipendenti per servizio rimuovono la necessità di coordinare un rilascio che coinvolga l'intera applicazione, abilitando in linea di principio rilasci più frequenti per singolo servizio.
- **Change Failure Rate:** due evidenze concrete supportano questa lettura. In primo luogo, due bug fix successivi sono fallimenti reali della pipeline stessa, introdotti e corretti nel corso dello sviluppo — citabili come "difetti individuati e corretti durante la fase sperimentale", con tempo di correzione molto rapido: il secondo risolve un problema introdotto lo stesso giorno dal fix precedente. In secondo luogo, e in modo più diretto, il gate reject-multi-service-impact (§4.2.1) agisce *a monte*, impedendo che una modifica multi-servizio venga anche solo mergiata: riducendo per costruzione l'ampiezza di ogni singola Merge Request che raggiunge la produzione, riduce il "blast radius" di un eventuale fallimento. Va ribadita qui la precisazione di §4.2.2: questa garanzia opera a livello di *Merge Request*, non di push su main, dove più MR possono confluire in un'unica esecuzione — un limite ripreso nel Capitolo 5, §5.1.2.

Va ribadito che, coerentemente con la priorità dichiarata, **non è necessario** che queste metriche siano raccolte in modo sistematico (Cap. 3, §3.2, pain point 7) perché il lavoro descritto in questa tesi sia considerato riuscito rispetto al suo obiettivo primario: la loro trattazione qui resta un framework di interpretazione qualitativo dei risultati ottenuti sulla produzione, non un requisito di completamento.

4.4 Sintesi del capitolo

I risultati presentati in questo capitolo si concentrano deliberatamente sulla catena che porta alla **produzione** (analisi d'impatto → build selettiva → immagine selettiva → deploy ECS selettivo, con il gate di rifiuto multi-servizio a presidio della disciplina che rende tutto ciò corretto), in linea con la priorità dichiarata in Cap. 1, §1.3. Con il rilascio reale in produzione questa catena non è più solo descritta ma **osservata**: la validazione sperimentale del capitolo culmina in un deploy automatico e selettivo effettivamente avvenuto, non in uno scenario ipotetico. Gli aspetti relativi a un ambiente UAT containerizzato e alla raccolta sistematica di metriche DORA, pur menzionati per completezza rispetto all'epic aziendale che ha originato questo lavoro, non sono oggetto di validazione sperimentale in questo capitolo poiché non ancora implementati, e la loro assenza non inficia le conclusioni qui presentate.

5

CAPITOLO 5

Considerazioni conclusive

5.1 Analisi critica

5.1.1 Benefici

Il percorso descritto in questa tesi porta a *optovera-be* una serie di benefici concreti, verificabili:

- **Rilascio del singolo microservizio, prima non possibile:** è la trasformazione più netta rispetto alla situazione di partenza. Prima di questo lavoro la build monolitica avveniva su *main*, ripubblicando tutte le nove immagini con lo stesso tag con il rilascio, seppur manuale, via Terraform allineando così l'intera applicazione (§3.1); rendere selettive analisi d'impatto, build e pubblicazione ha reso possibile aggiornare in produzione i soli servizi impattati, come osservato poi con la pubblicazione in produzione con *measurement* e *web*. Resta il limite, discusso poi più nel dettaglio in §5.1.2, per cui su *main* un push che accorpa più Merge Request produce comunque un rilascio multi-servizio: la garanzia del singolo servizio è imposta dal gate solo a livello di Merge Request.
- **Riduzione del lavoro superfluo in pipeline:** build, immagini Docker e (dove integrato) deploy sono oggi limitati ai servizi realmente impattati da una modifica, invece di essere eseguiti indiscriminatamente su tutti e nove i microservizi (Capitolo 3, §§3.4.1-3.4.4).
- **Automazione end-to-end del rilascio in produzione, dimostrata sul campo:** dal push su *main* all'aggiornamento del service ECS, senza alcun intervento manuale (*job deploy-prod*). Dunque non è più un beneficio solo potenziale: la pipeline ha eseguito un **rilascio reale e selettivo in produzione** (*measurement* e *web*, Cap. 4, §4.2.2). È il beneficio più direttamente allineato con l'obiettivo primario dichiarato in apertura di questa tesi, ed è, ad oggi, comprovato da un'esecuzione concreta.

- **Sicurezza migliorata nell'autenticazione verso AWS:** eliminazione delle credenziali statiche a favore di OIDC/Web Identity Federation, con credenziali temporanee scoped alla singola esecuzione di pipeline.
- **Disaccoppiamento fra ciclo di rilascio applicativo e ciclo di provisioning infrastrutturale:** Terraform resta responsabile del provisioning, mentre la pipeline applicativa gestisce autonomamente i rilasci di codice, così facendo rimane coerente con le pratiche DevOps discusse nel Capitolo 2, §2.2/§2.6.
- **Riduzione dell'accoppiamento nel codice applicativo:** il refactoring di disaccoppiamento dei moduli condivisi rende i servizi meno dipendenti, condizione abilitante perché la selettività della pipeline sia *corretta* e non solo apparente.
- **Governance del processo di rilascio, non solo tecnica:** uno degli ultimi interventi del percorso di tirocinio, il gate `reject-multi-service-impact` sposta, parte della disciplina "un solo microservizio impattato" dal codice (dove il refactoring di disaccoppiamento discusso in precedenza l'aveva resa possibile) al **processo**, impedendo esplicitamente in CI che una Merge Request ricongiunga più microservizi in un'unica modifica. È un beneficio che chiude concettualmente il cerchio fra disaccoppiamento architetturale e disaccoppiamento operativo, entrambi al servizio dell'obiettivo primario di deploy in produzione indipendenti e a basso rischio.
- **Metodologia di sviluppo incrementale e verificata:** ogni fase del percorso è stata introdotta, validata empiricamente con scenari dedicati in pipeline reale, e corretta quando emergevano output non attesi dai log dei singoli job esaminati. L'ultima fase (`reject-multi-service-impact`) mostra un ulteriore affinamento di questa metodologia: gli scenari di test usano un file marcatore dedicato invece di modificare file applicativi reali come nelle fasi precedenti, riducendo il rischio collaterale della validazione stessa (Capitolo 4, §4.2.1).

5.1.2 Limiti

Una riflessione quanto più scrupolosa sui limiti del lavoro è parte integrante del contributo di questa tesi. I limiti residui e non, ordinati per rilevanza rispetto all'obiettivo primario dichiarato (ovvero la produzione) coerentemente con l'impostazione già adottata nei Capitoli 3-4, sono i seguenti:

Rispetto all'obiettivo primario — il rilascio in produzione:

- **Su main, la selettività ha la granularità del push, non della Merge Request:** `detect-impact`, quando gira su main, calcola l'impatto sull'intero intervallo `$CI_COMMIT_BEFORE_SHA..$CI_COMMIT_SHA`, cioè su *tutte* le modifiche di un push. Se più Merge Request confluiscono in main in un unico avanzamento del ramo, `IMPACTED_SERVICES` diventa la loro unione e `deploy-prod` rilascia più servizi in un'unica esecuzione. È esattamente ciò che è accaduto nel deploy documentato

nel Cap. 4, §4.2.2 (measurement + web); lo stesso esito si presenterebbe, più in generale, ogniqualvolta più persone lavorino in parallelo su microservizi diversi e le rispettive Merge Request confluiscono nello stesso push su main. Il gate reject-multi-service-impact, che esisterebbe proprio per impedire modifiche multi-servizio, **non copre questo caso** perché gira only: merge_requests e non viene valutato sulle pipeline di main. Il risultato non compromette l'indipendenza del deploy (i servizi non toccati restano invariati), ma ciò significa che la regola di un solo microservizio impattato è garantita solo per il flusso di integrazione via singola MR, non per il push su main. È il limite più rilevante rispetto all'obiettivo primario, ora che è avvenuta la pubblicazione dei contributi discussi fino ad ora.

- **Le due reti di sicurezza sul percorso verso la produzione non sono bloccanti:** per quanto non sia stato oggetto principale del lavoro di tirocinio, c'è comunque da sottolineare che sia il Quality Gate SonarQube sia la suite di test end-to-end hanno allow_failure: true e non impediscono oggi a una modifica difettosa di raggiungere main e, potenzialmente, la produzione. È un'incoerenza diretta rispetto all'obiettivo di un rilascio in produzione affidabile, resa più stringente dal fatto che il rilascio è ora effettivamente automatico.

Sul piano architetturale, dei dati e dei confini fra repository:

- **Indipendenza non estesa al livello dei dati:** l'indipendenza raggiunta riguarda build, artifact e ciclo di rilascio, ma **non lo storage**. Dall'ispezione dei profili di produzione (application-prod.yml dei moduli -boot) risulta che sei servizi su nove (auth, measurement, web, consumer, migration, scheduler) puntano al medesimo database MongoDB main_db, e che analytics e migration condividono il database PostgreSQL analytics; RabbitMQ è anch'esso condiviso fra più servizi. Ne discende una conseguenza diretta sulla pipeline: una modifica allo schema o alla semantica dei dati condivisi, collocata nel codice di un singolo servizio, viene classificata da detect-impact come impatto su quel solo servizio, un potenziale **falso negativo**, perché la regola di fallback ALL è ancorata a opto-core/ e pom.xml, non ai contratti dati impliciti fra servizi che condividono un database. È il limite concettualmente più profondo: il disaccoppiamento è reale a livello di deployment, ma il sistema conserva un tratto di *distributed monolith* (Cap. 2, §2.1) sul piano dei dati. *(i nomi dei database sono condivisi e verificabili nel repository, ma l'host effettivo è iniettato a runtime via variabili — \${MONGODB_HOSTNAME}, \${POSTGRESQL_HOSTNAME} valorizzate nelle task definition ECS definite all'interno della repository dedicata a Terraform, non ispezionata e non oggetto del progetto di tirocinio; non è dimostrabile con certezza assoluta che si tratti della medesima istanza fisica, sebbene la condivisione del nome logico lo renda l'ipotesi di gran lunga più probabile.)*
- **Accoppiamento implicito col naming dell'infrastruttura Terraform:** un difetto risolto nel corso dello sviluppo ha mostrato che la pipeline applicativa deve conoscere esattamente il naming dei cluster ECS definiti nel repository Terraform separato, il disaccoppiamento perseguito in questo lavoro è quindi **parziale**: i cicli di

rilascio sono disaccoppiati temporalmente e operativamente, ma non l'informazione di naming, che resta un contratto implicito fra i due repository.

Su manutenibilità, copertura della selettività e completezza della valutazione:

- **Mappatura file→servizio mantenuta manualmente:** `detect-impact` richiede una riga `grep` dedicata per ogni nuovo servizio — non scala automaticamente all'aggiunta di microservizi, a differenza di altri strumenti dedicati come `Nx affected` o `Bazel`, discussi nel Capitolo 2, §2.5. Poiché questa mappatura determina direttamente cosa viene costruito e distribuito in produzione, è un limite con impatto diretto sull'obiettivo primario, non solo un dettaglio di manutenibilità.
- **Logica di selettività duplicata in più punti:** lo stesso controllo del microservizio impattato, è implementato sia in `builder.sh` (`build_selective`) sia inline nel job `build-and-push-images` (`process_image`), un rischio di disallineamento futuro (Capitolo 3, §3.4.4).
- **Pattern a tre rami (vuoto/ALL/selettivo) ripetuto** in quattro job diversi, senza fattorizzazione in un template GitLab comune (nessun `extends:YAML anchor` per questo scopo) — violazione del principio DRY, che rende la manutenzione più onerosa (Capitolo 3, §3.3).
- **Nessuna metrica raccolta in modo strumentato e continuativo: le durate riportate nel Capitolo 4, §4.3.1, provengono dalla lettura manuale della cronologia delle esecuzioni GitLab su singole esecuzioni campione, mentre le metriche DORA non sono oggi calcolate affatto.** Coerentemente con la priorità dichiarata in Cap. 1, §1.3, questo limite **non compromette** la valutazione del lavoro rispetto al suo obiettivo primario, ma resta un'area da completare per una valutazione quantitativa rigorosa (Capitolo 4, §4.3.1).
- **Copertura parziale dei "trigger" di rebuild globale:** la regola ALL scatta solo su modifiche a `opto-core/` o al `pom.xml` radice; modifiche a `builder.sh` o allo stesso `.gitlab-ci.yml` non sono coperte dalla regex di `detect-impact` — una modifica alla logica di build stessa non forza una validazione completa su tutti i servizi.
- **Nessun ambiente UAT containerizzato:** `deploy-stage` resta un deploy SSH non containerizzato dell'intera applicazione. Coerentemente con la priorità dichiarata, questo limite è riconosciuto ma **non centrale** per la valutazione del lavoro svolto.

5.1.3 Confronto con la soluzione iniziale

La Tabella 5.1 riassume, aspetto per aspetto, il passaggio dalla pipeline iniziale a quella selettiva.

La colonna "Dopo" descrive lo stato consolidato in `main` alla data di stesura di questa tesi: le due trasformazioni più rilevanti rispetto all'obiettivo primario (deploy automatico,

Aspetto	Prima (Capitolo 1, §1.2)	Dopo (Capitoli 3-4)
Build	Sempre tutti i 9 servizi	Solo i servizi impattati (fallback ALL su modifica condivisa)
Immagini Docker	Tutte pubblicate ad ogni pipeline su main	Solo quelle dei servizi impattati
Deploy produzione	Manuale via Terraform (SHA copiato a mano dal log del job main → terraform apply)	Automatico via deploy-prod, per servizio (in main, dimostrato in produzione)
Rilascio del singolo microservizio	Non possibile: build monolitica → ogni apply muove tutti i servizi	Possibile per i servizi impattati (garanzia del singolo servizio sulle MR; §5.1.2)
Autenticazione AWS	Credenziali statiche	OIDC, credenziali temporanee
Accoppiamento codice fra servizi	Moduli condivisi con classi esclusive (project-service)	Classi esclusive migrate ai rispettivi moduli -boot
Ciclo rilascio applicativo vs infrastruttura	Intrecciati (deploy tramite Terraform)	Disaccoppiati (deploy applicativo autonomo dalla pipeline)
Disciplina "un servizio per modifica"	Solo convenzionale, non imposta	Imposta in CI da reject-multi-service-impact sulle MR (in main; non copre i push multi-MR, §5.1.2)
Indipendenza a livello dati	Database condivisi	Invariato: storage tuttora condiviso (main_db, Postgres analytics), fuori dallo scopo del lavoro (§5.1.2)

Tabella 5.1: Confronto sintetico fra la pipeline iniziale e la pipeline selettiva.

disciplina imposta) sono integrate nel branch principale, e il deploy automatico è stato osservato in un rilascio reale in produzione (Cap. 4, §4.2.2). Restano i limiti discussi in §5.1.2 — in particolare la granularità del push su main e la persistenza dell'accoppiamento a livello dati — che qualificano, senza smentirle, le conquiste riassunte nella colonna "Dopo".

5.2 Considerazioni finali

Il lavoro descritto in questa tesi trasforma la pipeline CI/CD di un monorepo backend Spring Boot da un modello monolitico a un modello di **build e deploy selettivi per servizio**, affrontando in parallelo quattro dimensioni: tecnica (analisi d'impatto, selettività Maven/Docker/ECS), architetturale (refactoring per ridurre l'accoppiamento del codice), di sicurezza (autenticazione OIDC) e, nella fase più recente, di **governance del processo** (il gate reject-multi-service-impact): non basta rendere la selettività tecnicamente possibile, occorre anche impedire in CI che venga aggirata.

Coerentemente con la priorità dichiarata fin dal Capitolo 1, il criterio con cui questo lavoro va giudicato è, in primo luogo, la sua capacità di rendere il rilascio in **produzione** indipendente, automatico e affidabile per singolo microservizio — un obiettivo che, con l'integrazione dei due branch finali in main e il rilascio reale in produzione, è oggi **raggiunto e dimostrato sul campo**, non più solo implementato e validato in isolamento. Restano da irrobustire aspetti reali (la granularità del push su main, i gate di qualità non bloccanti, l'accoppiamento a livello dati, §5.1.2), ma la conquista centrale del lavoro è stata osservata in produzione. Un ambiente UAT containerizzato e la raccolta sistematica di metriche DORA restano traguardi validi e discussi in questa tesi, ma esplicitamente secondari: la loro assenza non intacca la valutazione complessiva del lavoro svolto.

5.3 Possibili evoluzioni

Gli spunti seguenti sono proposte di questa tesi, non iniziative già pianificate dal team salvo conferma esplicita, e sono ordinati per priorità rispetto all'obiettivo primario di produzione. La proposta un tempo più urgente — l'integrazione dei due branch finali in main — è **già stata realizzata** e non compare più in questo elenco.

- **Estendere la disciplina "un servizio per push" anche a main:** oggi reject-multi-service-impact gira only: merge_requests e non presidia i push diretti su main, dove più Merge Request possono confluire in un'unica pipeline e quindi in un unico deploy multi-servizio (§5.1.2, Cap. 4 §4.2.2). Possibili direzioni: valutare l'impatto per singolo *merge commit* invece che sull'intero arco del push, oppure estendere il gate a main con una semantica adeguata al contesto post-merge. È il passo che rende la promessa di un solo microservizio impattato vera end-to-end, non solo sul flusso di integrazione via MR.

- **Rendere bloccanti il Quality Gate SonarQube e i test end-to-end**, valutando se eseguirli anche sulle Merge Request e non solo su main e coerentemente attuare un lavoro preciso e considerevole sui test, chiuderebbe la lacuna più diretta sulla affidabilità del percorso verso la produzione, tanto più rilevante ora che il rilascio è effettivamente automatico.
- **Estendere l'analisi d'impatto ai contratti dati condivisi**: poiché più servizi condividono lo stesso database (§5.1.2), una modifica allo schema dei dati collocata in un singolo servizio può impattarne altri senza che detect-impact lo rilevi. Una direzione possibile è marcare come "condivisi" (fallback allargato, non necessariamente ALL) i percorsi che definiscono modelli/entità persistite su database comuni, analogamente a quanto già fatto per opto-core/ e pom.xml. Affronta il limite concettualmente più profondo del lavoro, l'indipendenza non ancora estesa al livello dei dati.
- **Introdurre una cache delle dipendenze Maven e dei layer Docker**, per ridurre il tempo che separa un commit dal rilascio in produzione, oggi penalizzato da download e build sistematicamente "a freddo" (incluso il download ripetuto dell'agent OpenTelemetry ad ogni build dell'immagine, Cap. 2 §2.4).
- **Validare automaticamente il naming cross-repository** fra optovera-be e terraform-optovera (ad esempio un contract test che verifichi che i cluster ECS attesi dalla pipeline esistano realmente), per prevenire in anticipo difetti di naming analoghi a quello risolto nel corso di questo lavoro.
- **Adottare uno strumento dedicato di dependency graph** (Nx, Bazel, o anche solo mvn dependency:tree analizzato programmaticamente) per derivare automaticamente la mappatura file→servizio, invece di mantenerla a mano — riduce un rischio che incide direttamente su cosa viene costruito e distribuito in produzione.
- **Fattorizzare il pattern a tre rami** in un job/template GitLab riutilizzabile, riducendo la duplicazione oggi presente in quattro job diversi.
- **Strategie di deploy più sofisticate su ECS**: blue/green o canary al posto del rolling update di default di update-service, per ridurre ulteriormente il rischio nei rilasci automatizzati.
- *(Secondario, non bloccante)* **Introdurre un ambiente UAT containerizzato su Docker**, distinto dall'attuale deploy-stage via SSH.
- *(Secondario, non bloccante)* **Raccolta sistematica di metriche DORA** integrata nella pipeline (ad esempio un job che pubblica durata ed esito su una dashboard dedicata).

Glossario

Il glossario raccoglie i termini chiave indicati in apertura della tesi, con una definizione sintetica e il rimando al paragrafo in cui ciascuno è trattato in modo esteso.

CI/CD — acronimo di Continuous Integration / Continuous Delivery. Insieme di pratiche che automatizzano integrazione, verifica e rilascio del software: ogni modifica al codice viene compilata e verificata automaticamente (CI) e resa rilasciabile in modo ripetibile verso gli ambienti di esecuzione (CD). Trattato in §2.2.

Microservizi — stile architetturale in cui un'applicazione è realizzata come insieme di servizi autonomi, ciascuno responsabile di una capacità funzionale, con ciclo di vita e rilascio indipendenti. Si contrappone all'architettura monolitica. Trattato in §2.1.

Monorepo — organizzazione del codice sorgente in cui più servizi o moduli, anche indipendenti fra loro, risiedono in un unico repository condiviso. Semplifica riuso e versionamento trasversale, ma rende non banale distinguere quali servizi siano effettivamente interessati da una singola modifica. Contesto in §2.6.

Analisi d'impatto — in questa tesi, la fase di pipeline (impact detection) che, confrontando le modifiche introdotte con la mappa delle dipendenze fra moduli, determina l'insieme dei microservizi effettivamente coinvolti, propagato ai job successivi nella variabile IMPACTED_SERVICES. Implementazione in §3.4.1.

Build selettiva — strategia di compilazione che limita la build ai soli moduli impattati da una modifica e alle loro dipendenze upstream, anziché ricostruire l'intero progetto. Nel progetto è realizzata tramite le opzioni Maven `-pl` e `-am`. Trattata in §2.5 e §3.4.2.

Deploy indipendente — possibilità di rilasciare in produzione un singolo microservizio senza coinvolgere né bloccare gli altri, condizione resa possibile dall'analisi d'impatto e dalla containerizzazione per servizio. Risultati in §4.3.2.

GitLab CI/CD — piattaforma di automazione delle pipeline integrata in GitLab, configurata tramite il file dichiarativo `.gitlab-ci.yml` e organizzata in stage e job con dipendenze esplicite (needs:) che abilitano l'esecuzione parallela. Trattata in §2.7.1.

Docker — tecnologia di containerizzazione che impacchetta un'applicazione e le sue dipendenze in un'immagine eseguibile e isolata, garantendo la coerenza del comportamento fra ambiente di sviluppo e di produzione. Trattata in §2.4 e §2.7.3.

Amazon ECS — Elastic Container Service, servizio gestito di orchestrazione dei container su AWS: esegue i container secondo task definition immutabili e ne governa aggiornamento e ciclo di vita tramite rolling deployment. Trattato in §2.7.3.

Terraform — strumento di Infrastructure as Code che descrive l'infrastruttura in forma dichiarativa e la riconcilia con lo stato desiderato, rendendo il provisioning versionabile e riproducibile. Trattato in §2.4 e §2.7.4.

OIDC — OpenID Connect, protocollo di autenticazione federata utilizzato dalla pipeline per ottenere credenziali temporanee verso il cloud provider, eliminando la necessità di conservare chiavi di accesso statiche nelle variabili di progetto. Trattato in §2.3.

DevOps — approccio organizzativo e culturale che integra sviluppo ed esercizio del software in un unico flusso di responsabilità condivisa, con l'automazione come strumento abilitante e la misurabilità del processo come criterio di valutazione. Trattato in §2.2 e §2.8.

Bibliografia

Questa sezione raccoglie, in stile autore-data, tutti i riferimenti bibliografici citati nella tesi. Sotto ciascuna voce è indicato il capitolo e il paragrafo in cui la fonte è richiamata. Le fonti non bibliografiche (repository di progetto, file di configurazione, testimonianza diretta dell'autrice) sono raccolte separatamente nella sezione successiva.

Testi e articoli scientifici

Documentazione tecnica ufficiale

Fonti interne al progetto (non bibliografiche, per riferimento incrociato)

Non vanno in bibliografia in senso stretto, ma sono citate a supporto delle affermazioni tecniche nei capitoli e vanno mantenute coerenti con esse:

- File di configurazione dei profili di produzione dei moduli (`project-<servizio>-boot/src/main/resources/application-prod.yml`), citati in Cap. 2 §2.1 e Cap. 5 §5.1.2 a supporto dell'affermazione sulla condivisione dei database fra servizi.
- Dockerfile radice del repository, citato in Cap. 2 §2.4 per la struttura single-stage dell'immagine e il download dell'agent OpenTelemetry.
- Repository `terraform-optovera` (Infrastructure as Code della produzione, distinto dal backend), citato **per solo ruolo** in Cap. 3 §3.1 (file di variabili locali aggiornato manualmente col tag immagine, seguito da `terraform apply`), Cap. 2 §2.7.4, Cap. 5 §5.1.2/§5.1.3 e §5.3 (punto 5, validazione automatica del naming cross-repository). **Non ispezionato in questa tesi:** le affermazioni si limitano al suo ruolo nel flusso di rilascio.
- Testimonianza sul **processo di rilascio manuale pre-esistente** (copia dello SHA dal log del job di build su main → aggiornamento a mano del tag in `terraform-optovera` → `terraform apply`), eseguito durante il tirocinio prima dell'automazione descritta nel Cap. 3. A supporto dell'affermazione, in Cap. 1 §1.4 / Cap. 3 §3.1 / Cap. 5 §5.1.1, che prima di questo lavoro non era possibile rilasciare un singolo microservizio.

